

第03周：边缘与轮廓

从“看起来像边缘”到精确边缘检测，从零散像素到有意义的形状



上周回顾



示例：盒式滤波器

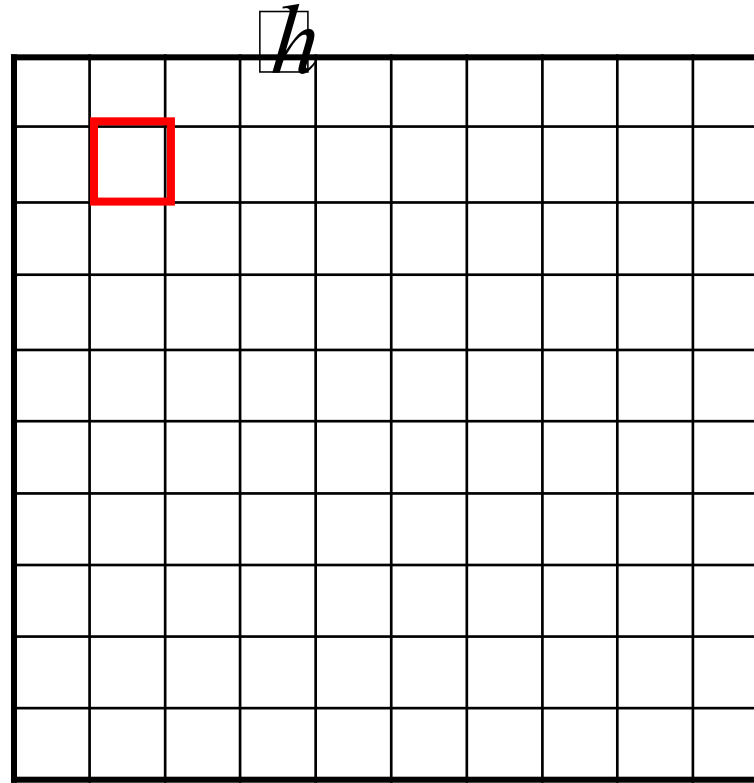
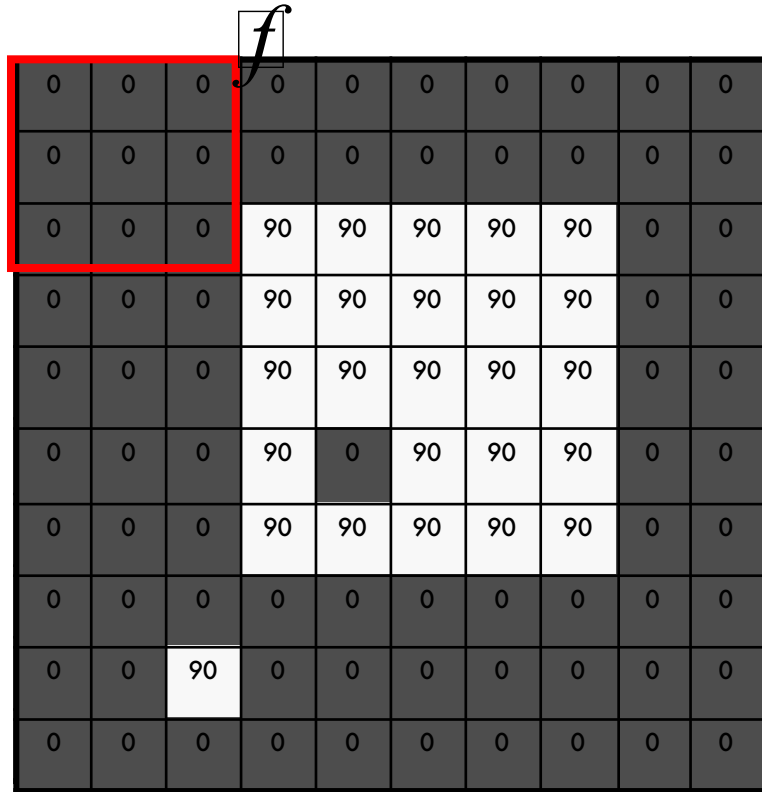
$[\cdot, \cdot]g$

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



☐ ,

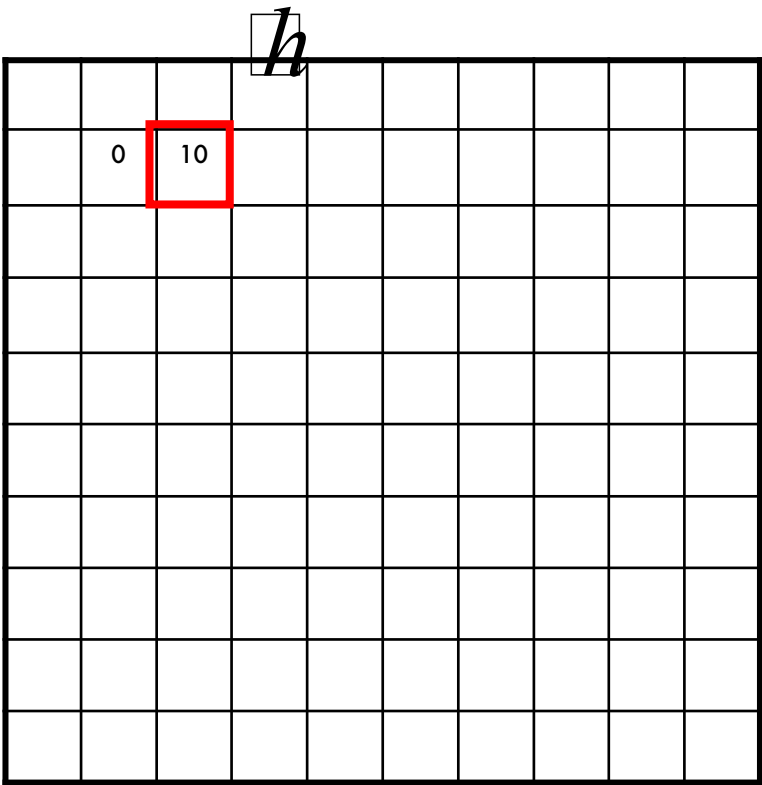
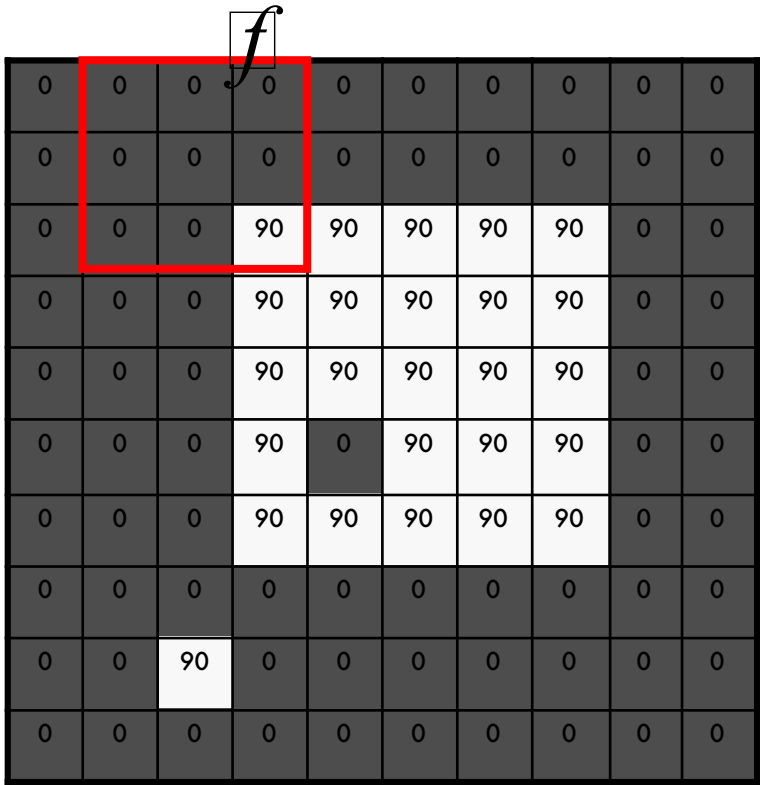


$$[\quad, \ln k m f l k g n m h \quad]^{+} \quad, \quad [\quad, \quad]^{+}$$

图像滤波

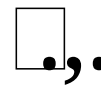
$[\cdot, \cdot]_g \frac{1}{9}$

1	1	1
1	1	1
1	1	1



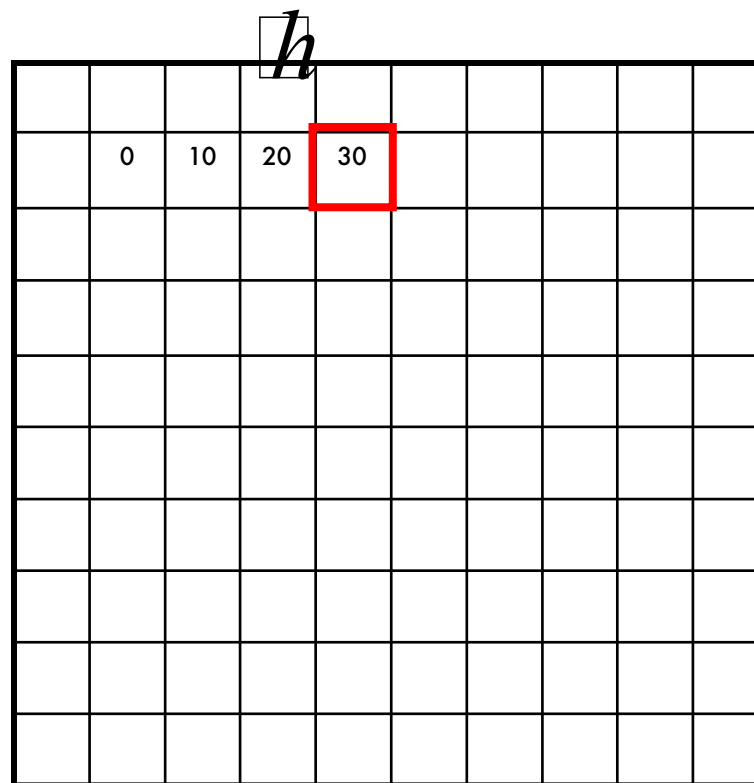
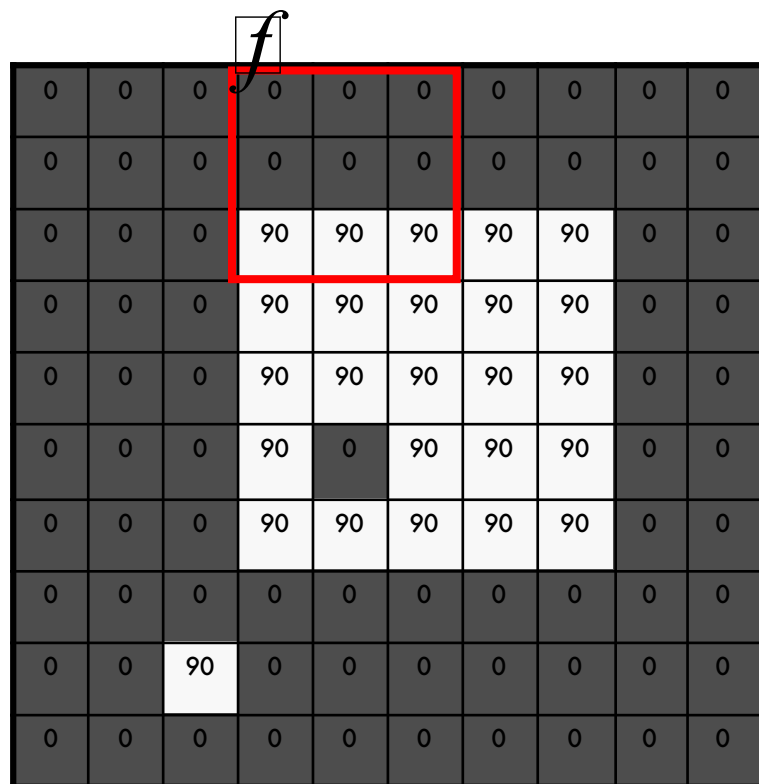
$[\cdot, \cdot]_g = [\cdot, \cdot]_h + [\cdot, \cdot]_f$

□,;



致谢: S. Seitz

□,;

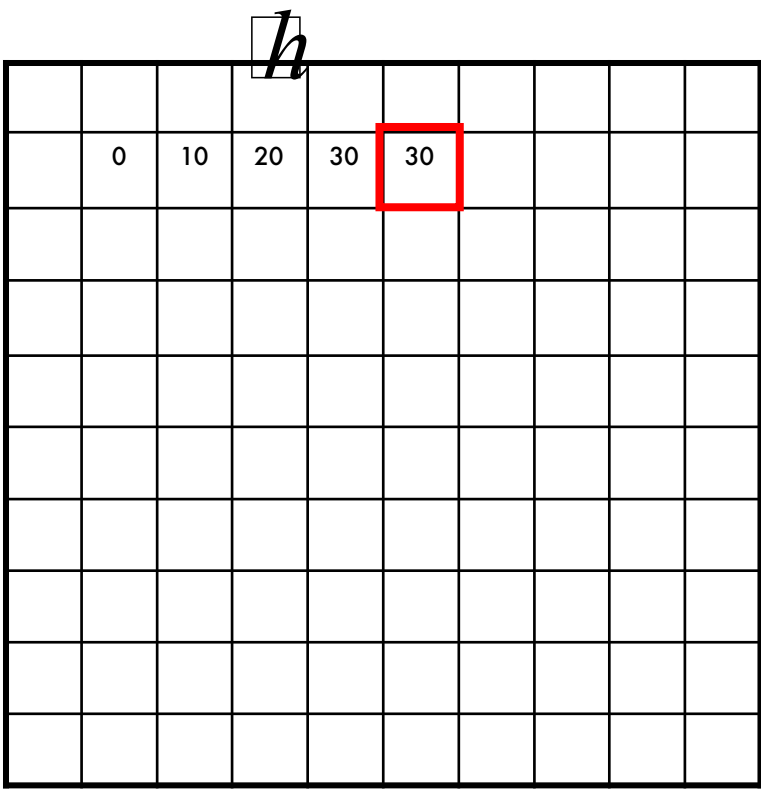
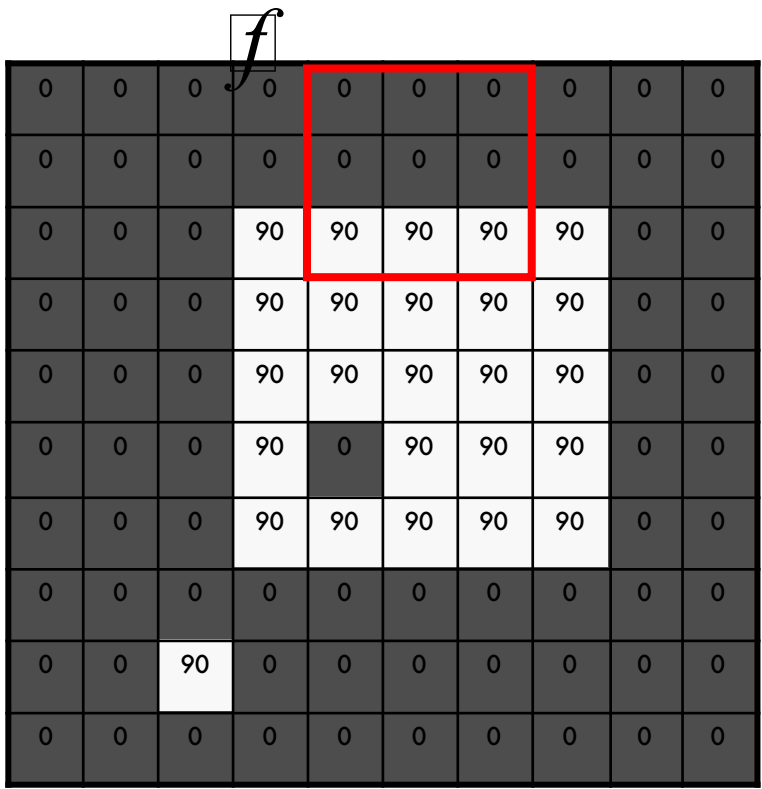


来源: S. Seitz

图像滤波

$[\cdot, \cdot]_g \frac{1}{9}$

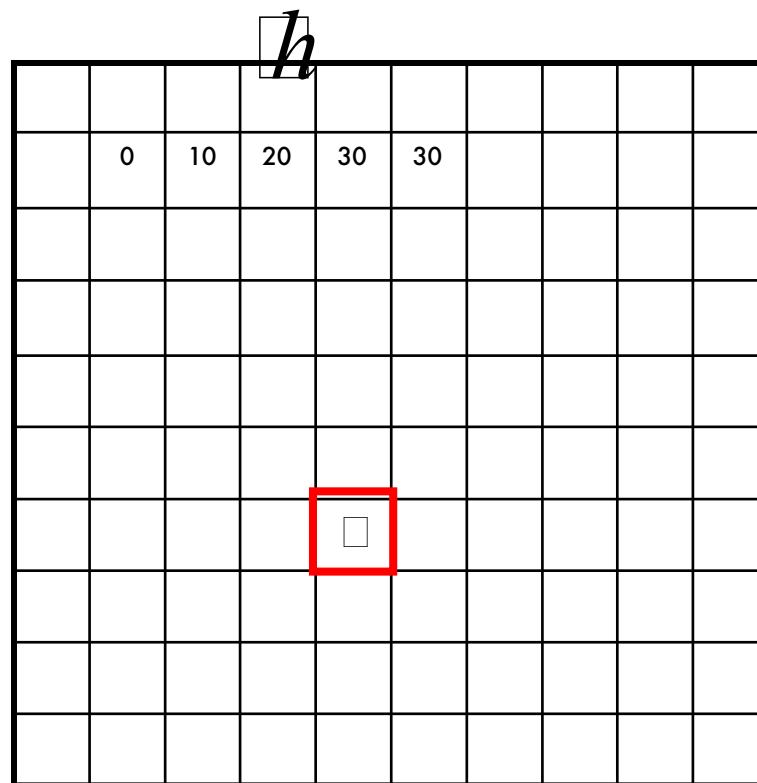
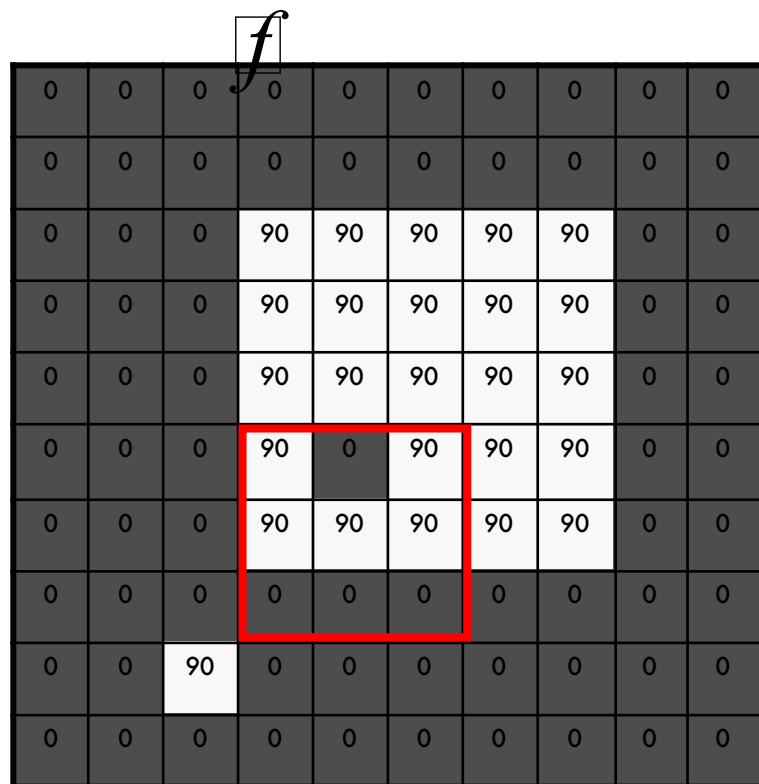
1	1	1
1	1	1
1	1	1



$$[\cdot, \cdot]_g = [\cdot, \cdot]_f + [\cdot, \cdot]_h$$

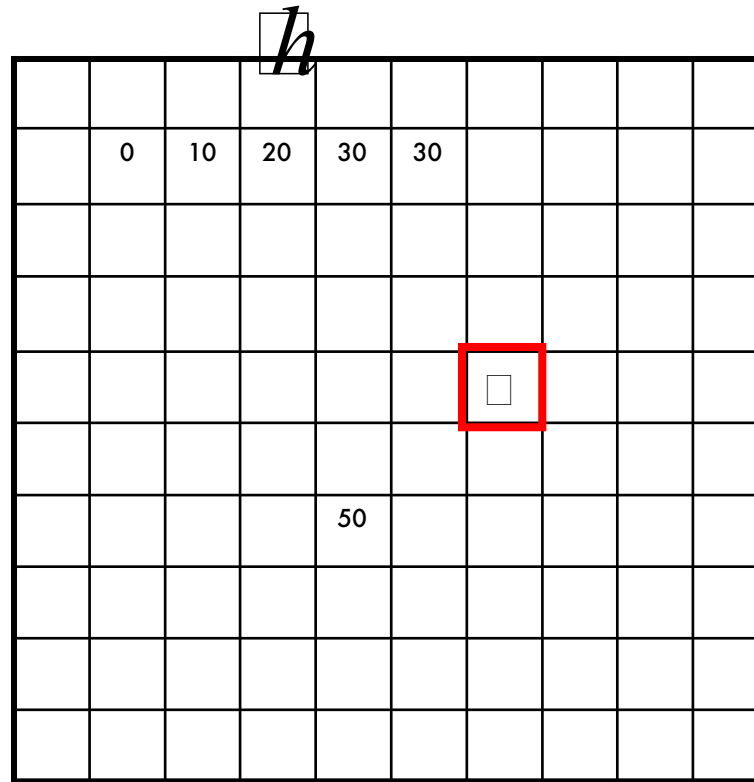
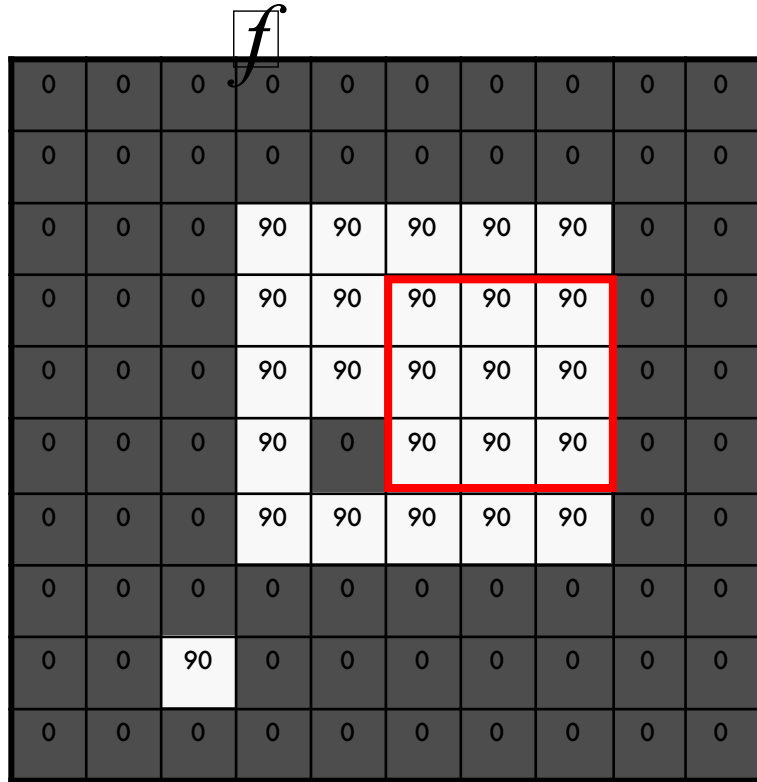
□, •

□, □



图片来源: S. Seitz

☐ ,



来源: S. Seitz

图像滤波

$$[\cdot, \cdot]_g \frac{1}{9}$$

1	1	1
1	1	1
1	1	1



f

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

h

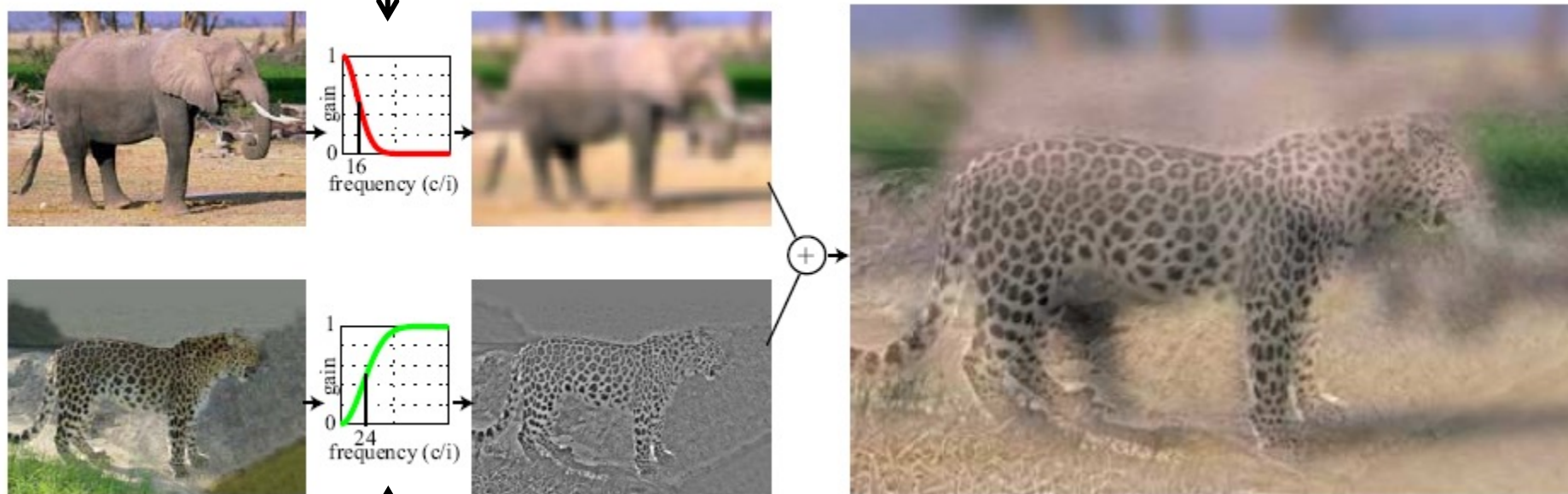
	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$[\cdot, \cdot]_g = [\cdot, \cdot]_f + [\cdot, \cdot]_h$$

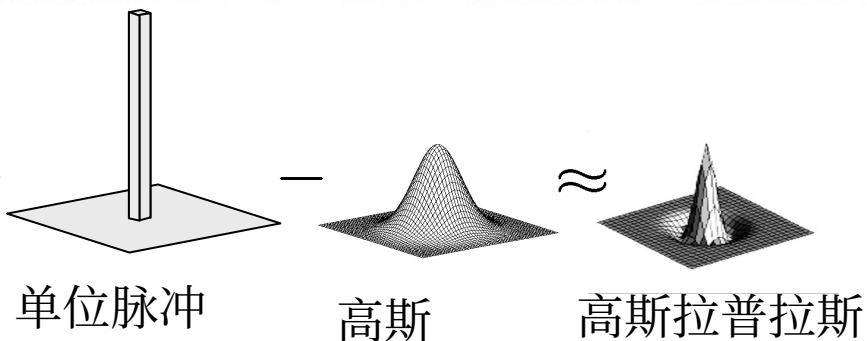
应用：混合图像_A

高斯滤波器！

· Oliva, A. Torralba, P.G. Sch
[a_bris_b_a_y_SbGGgAPa_A66G](#)

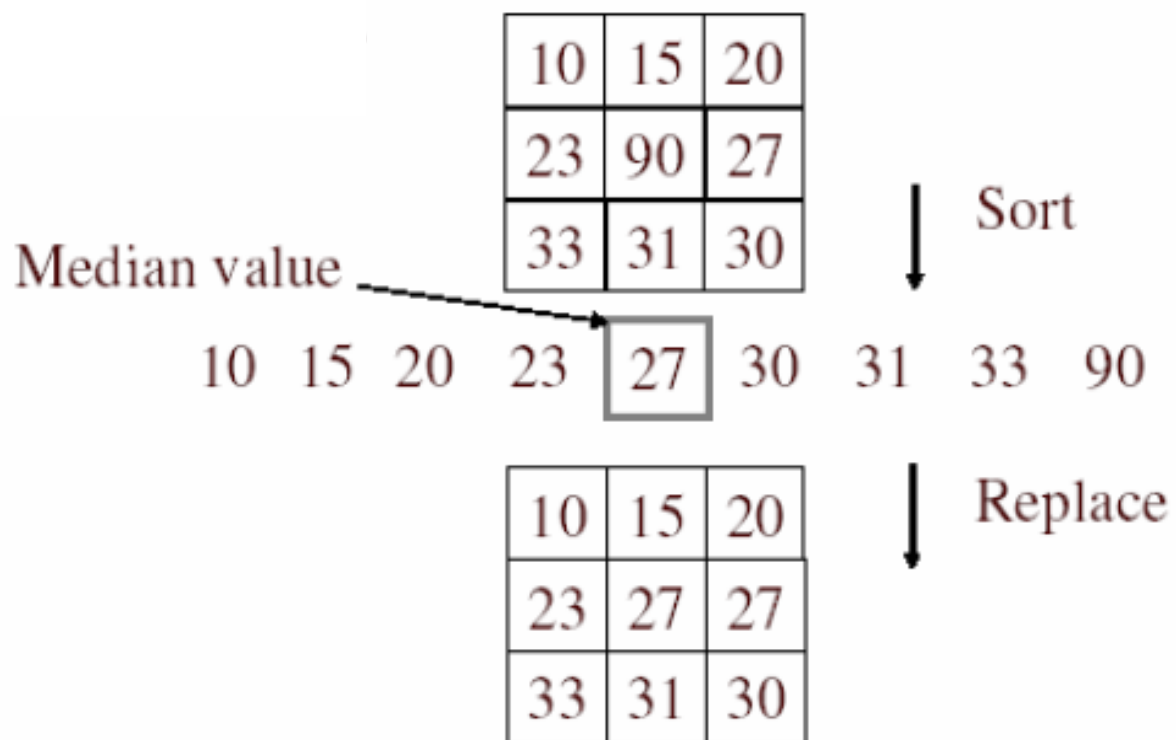


拉普拉斯滤波器！



另一种思路：中值滤波

- 一个中值滤波器在一个窗口上操作，通过选择窗口内的中值强度



- 中值滤波是线性的吗？

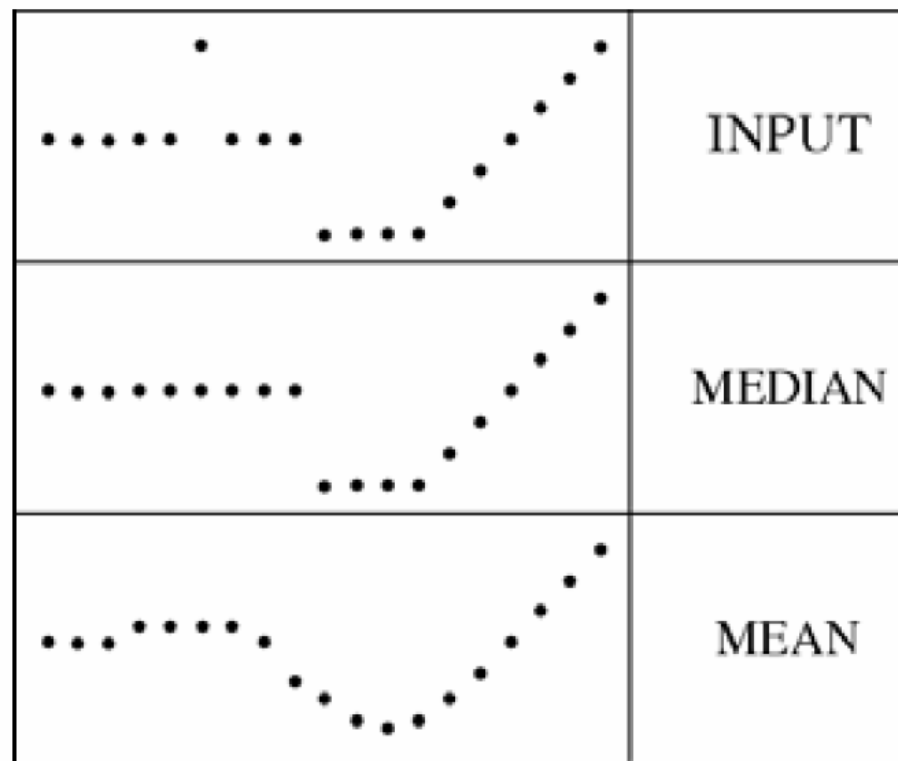


中值滤波器

- 中值滤波相比高斯滤波有什么优势?

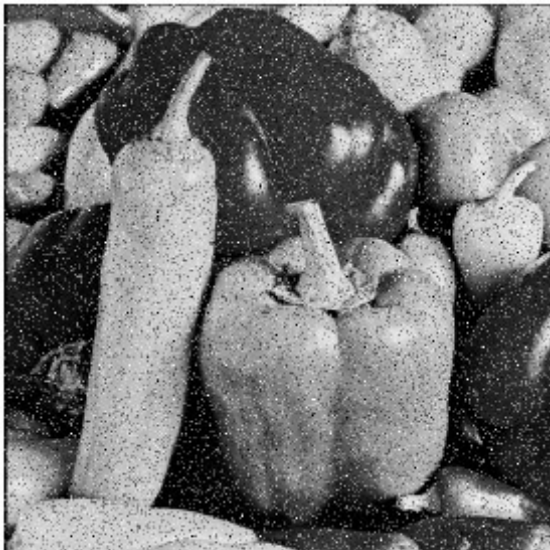
□ 对离群值的鲁棒性

filters have width 5 :

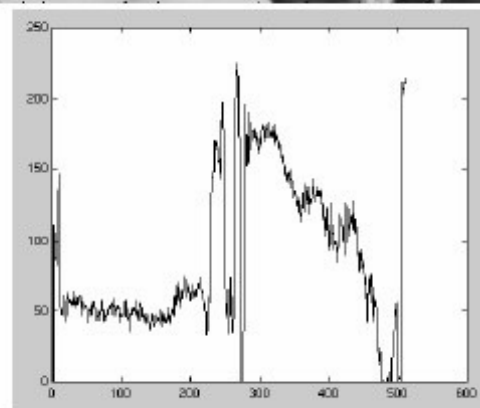
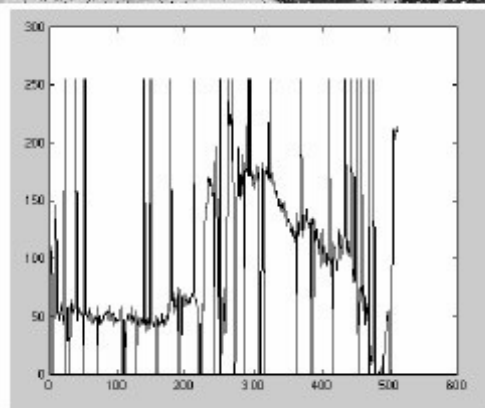


中值滤波器

椒盐噪声



中值滤波后

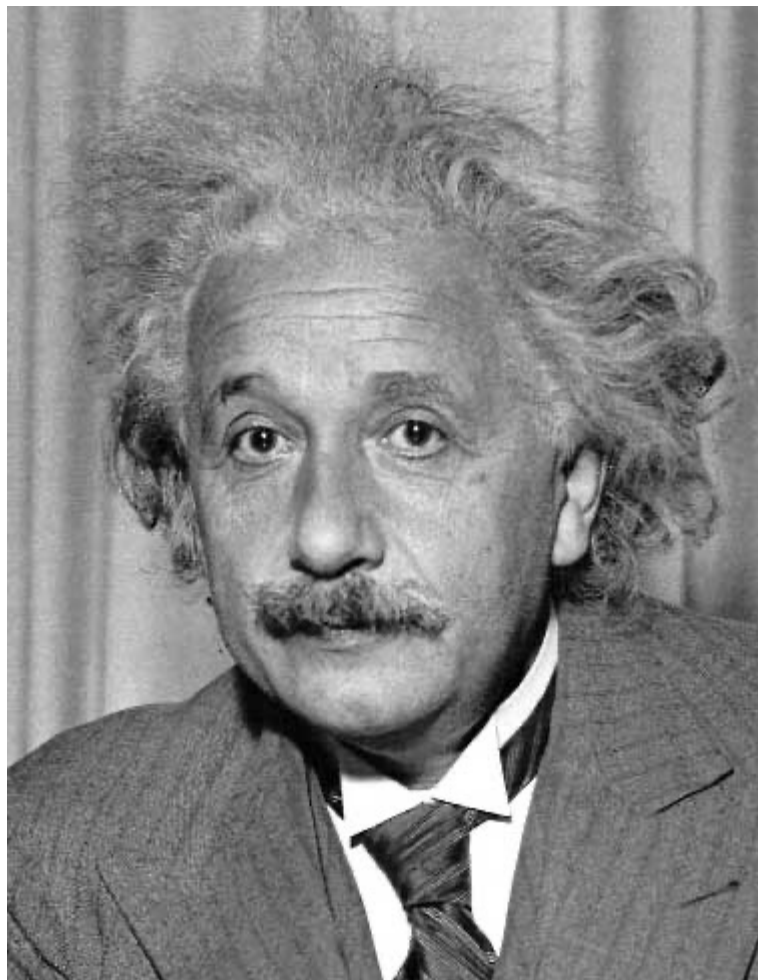


Python: `scipy.ndimage.median_filter(image, size)`



模板匹配

- 目标：在图像中查找
- 主要挑战：两个图像块之间什么才是好的相似度或距离度量？



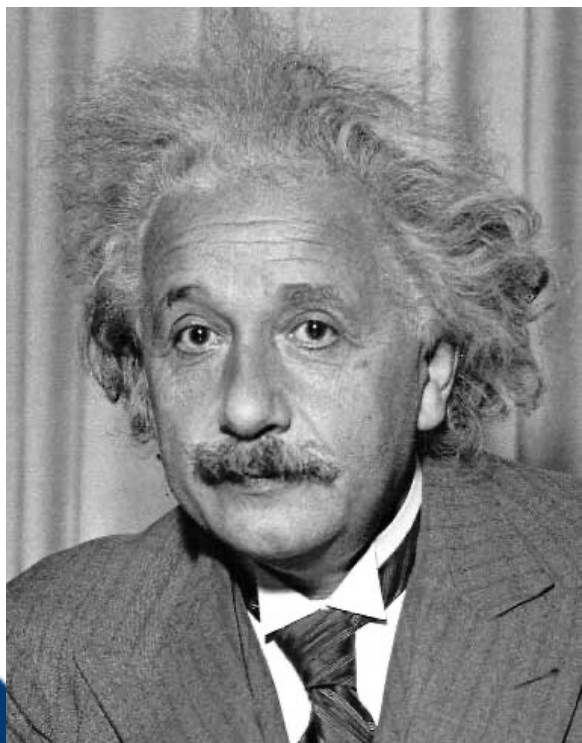
基于滤波器的匹配

- 目标：在图像中查找
- 方法 0：用眼部图块对图像进行滤波

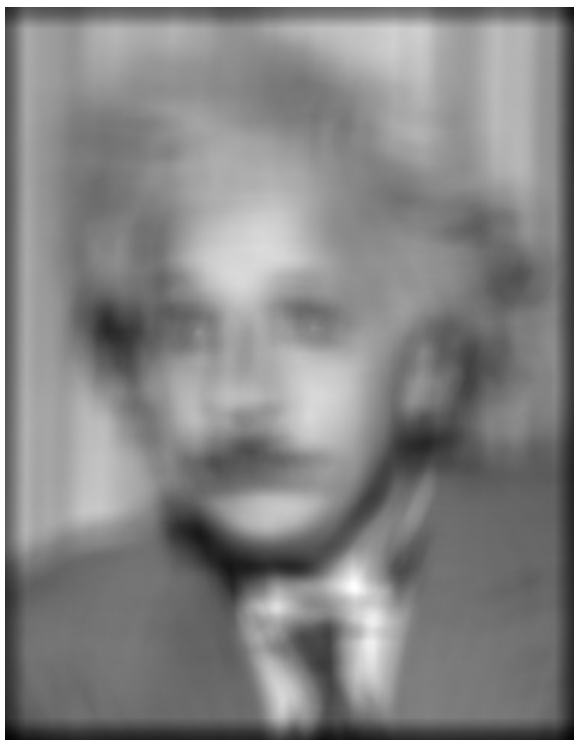
$$h[m,n] = \sum_{k,l} f[k,l] \text{ im}[m+k,n+l]$$

im mimage
f mfilter

出了什么问题？



输入

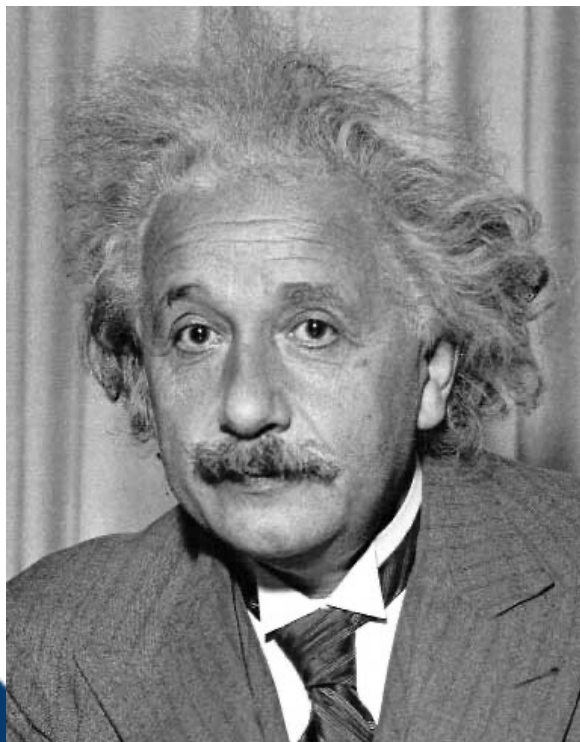


滤波后图像

使用滤波器进行匹配

- 目标：在图像中查找
- 方法 1：用零均值 eye 对图像进行滤波

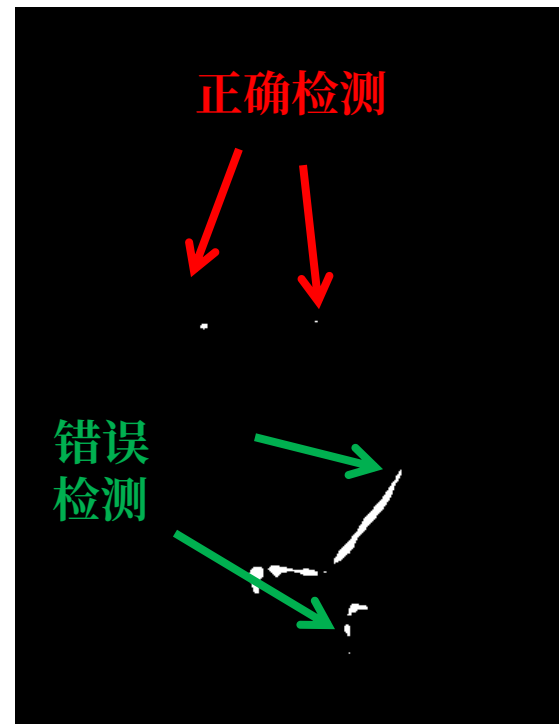
$$h[m,n] = \sum_{k,l} (f[k,l] - \underbrace{f}_{\text{滤波器 } f \text{ 的均值}}) im[m+k, n+l]$$



输入



滤波后的图像（已缩放）



阈值化图像

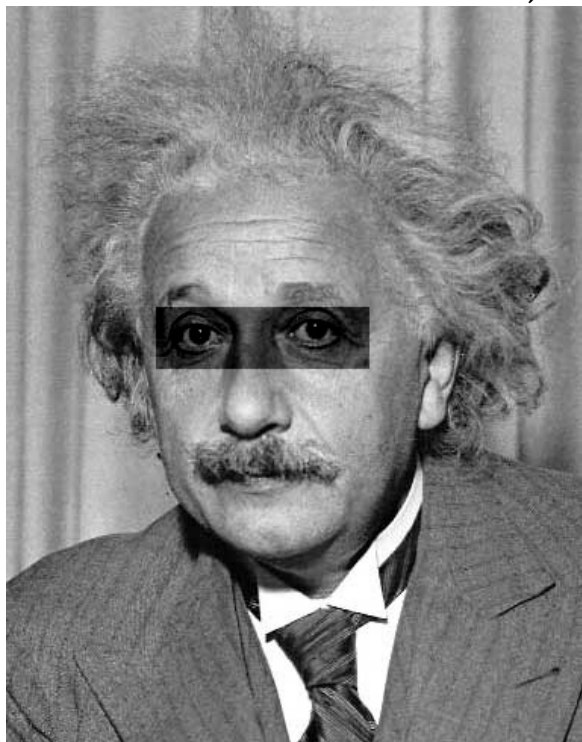
使用滤波器进行匹配

- 目标：在图像中查找
- 方法 2：距离平方和 (SSD)

$$h[m, n] = \sum_{k, l} (f[k, l] - im[m + k, n + l])^2$$

SSD 的潜在缺点是什么？

对强度偏移敏感



输入



1- sqrt(SSD)

使用滤波器进行匹配

- 目标：在图像中查找
- 方法 3：归一化互相关

$$h[m, n] = \frac{\begin{array}{c} \text{mean template} \\ \downarrow \\ \sum_{k,l} (f[k, l] - \bar{f}) \end{array} \begin{array}{c} \text{mean image patch} \\ \downarrow \\ \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}}) \end{array}}{(\sum_{k,l} (f[k, l] - \bar{f})^2 \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}})^2)^{0.5}}$$

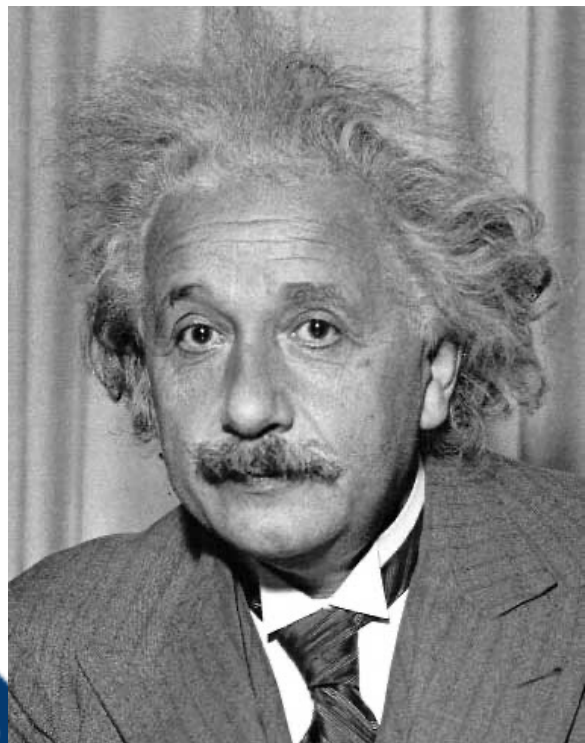
(除以模板与图像块的标准差的乘积)



Python: `cv2.matchTemplate(im, template, cv2.TM_CCOEFF_NORMED)`

使用滤波器进行匹配

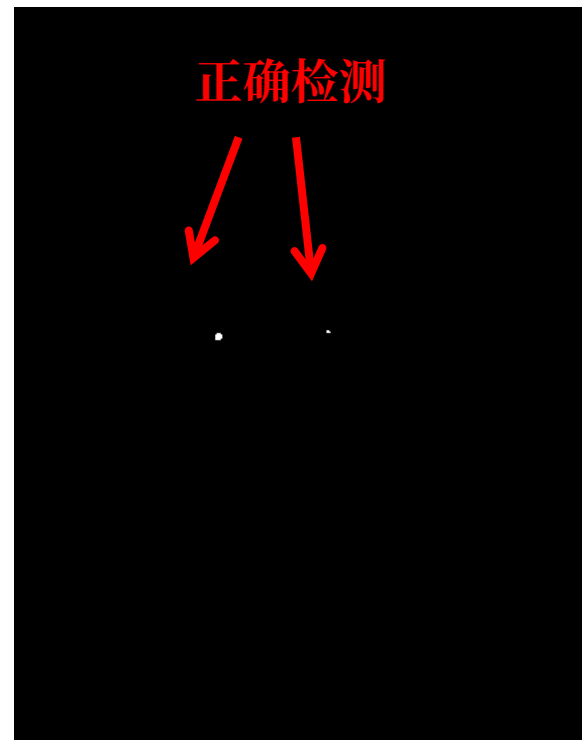
- 目标：在图像中查找
- 方法 3：归一化互相关



输入



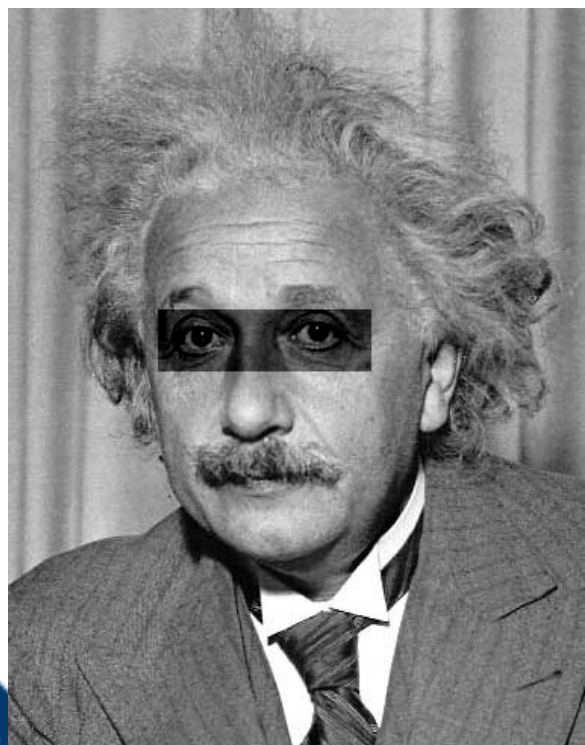
归一化互相关



归一化图像

使用滤波器进行匹配

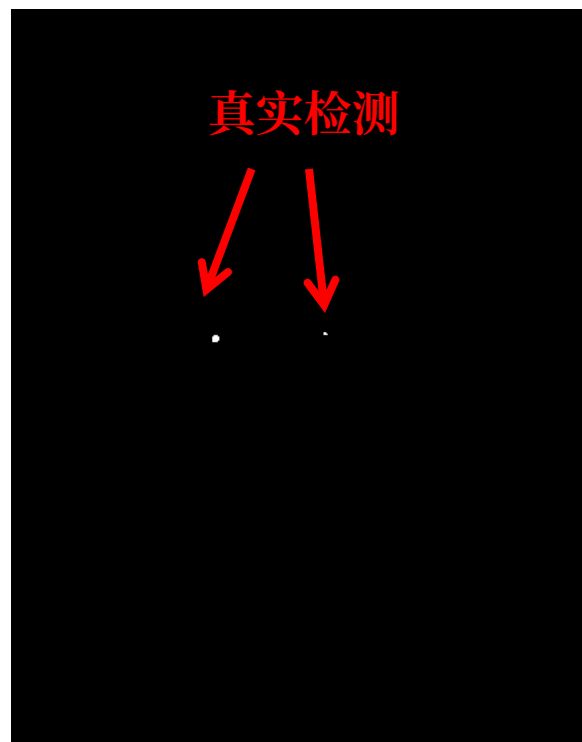
- $\square$ oa $\square$ mind  in i $\square$ a $\square$ e
- 方法 3: 归一化互相关



输入



归一化X-互相关



阈值化图像

硬币计数挑战

有多少枚硬币？计算机能数它们吗？



挑战

有多少枚硬币？

- 你可以用肉眼数出来。但如
- 果有 **1,000** 枚呢？或者
- **10,000** 枚呢？

目标：到今天结束时，我们将让计算机自动数出它们。



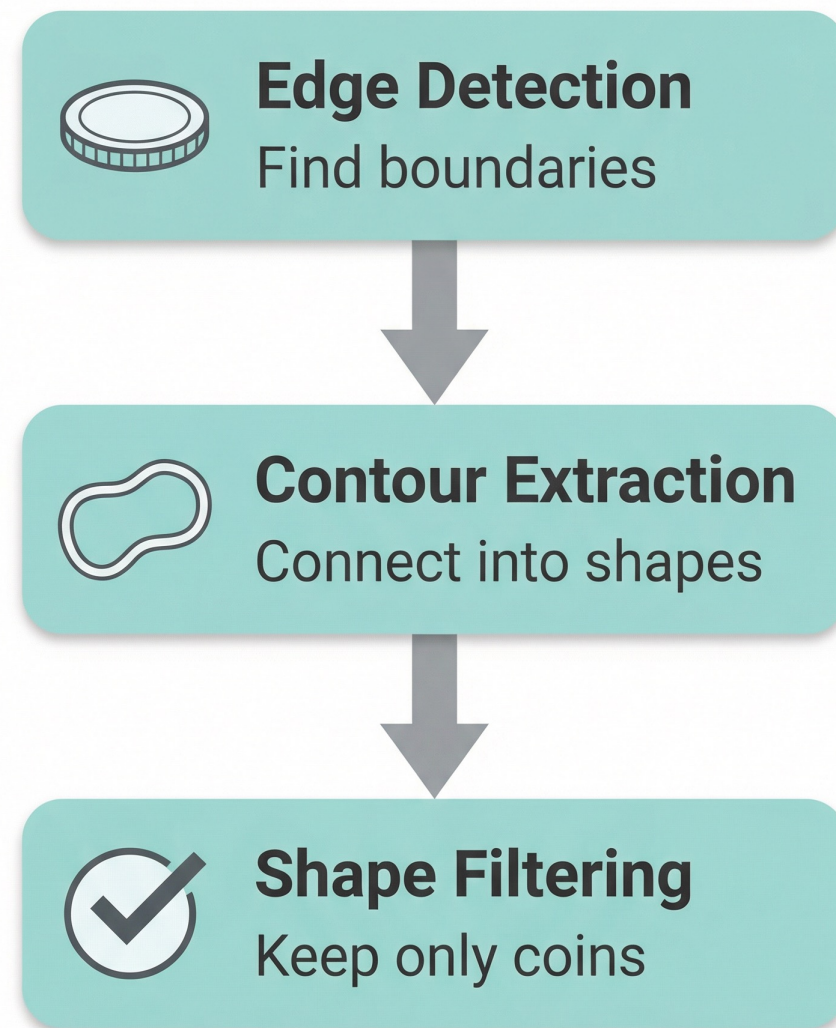
计算机需要做什么？

分解问题

1. 针对每枚硬币，找出边界
2. 将各个硬币分离
3. 对它们进行计数

这需要：

- 边缘检测 → 找到边界
- 轮廓提取 → 连接成形状



让我们尝试已有知识

尝试：高频 + Contour Detection

我们在第 2 周已经知道如何提取“边缘”：

```
高_频 = coins - GaussianBlur(coins)
```

然后在高频图像上查找轮廓.....



高频 $\approx$ 边缘?

原始	高斯模糊后	高频
		
锐利细节	低通：平滑	看起来像边缘！但它精确吗？

它能奏效吗？



高频 + 轮廓 = 失败

高频图像	轮廓检测结果
	Count: 9800 
噪声、断开的边缘	预期 12 枚硬币 → 得到 200+ 轮廓 ❌

为什么会失败？

高频边缘的三个问题

问题	描述
不连续	边缘在许多点处断开——不连续
噪声过多	随机变化会产生虚假的“边缘”
过粗	边缘跨度为5–10像素宽，并不精确

高频图像给予我们“边缘的感觉”，但它无法告诉我们**哪个确切的像素是边缘**。



今日旅程

“从模糊的边缘感觉到→ 精确的边缘位置”

开始	学习	应用
高频失败	Sobel & Canny	正确清点硬币！
$\approx$ 边缘	= 边缘	→ 轮廓 → 计数



第3部分

边缘背后的数学

导数、梯度及其重要性



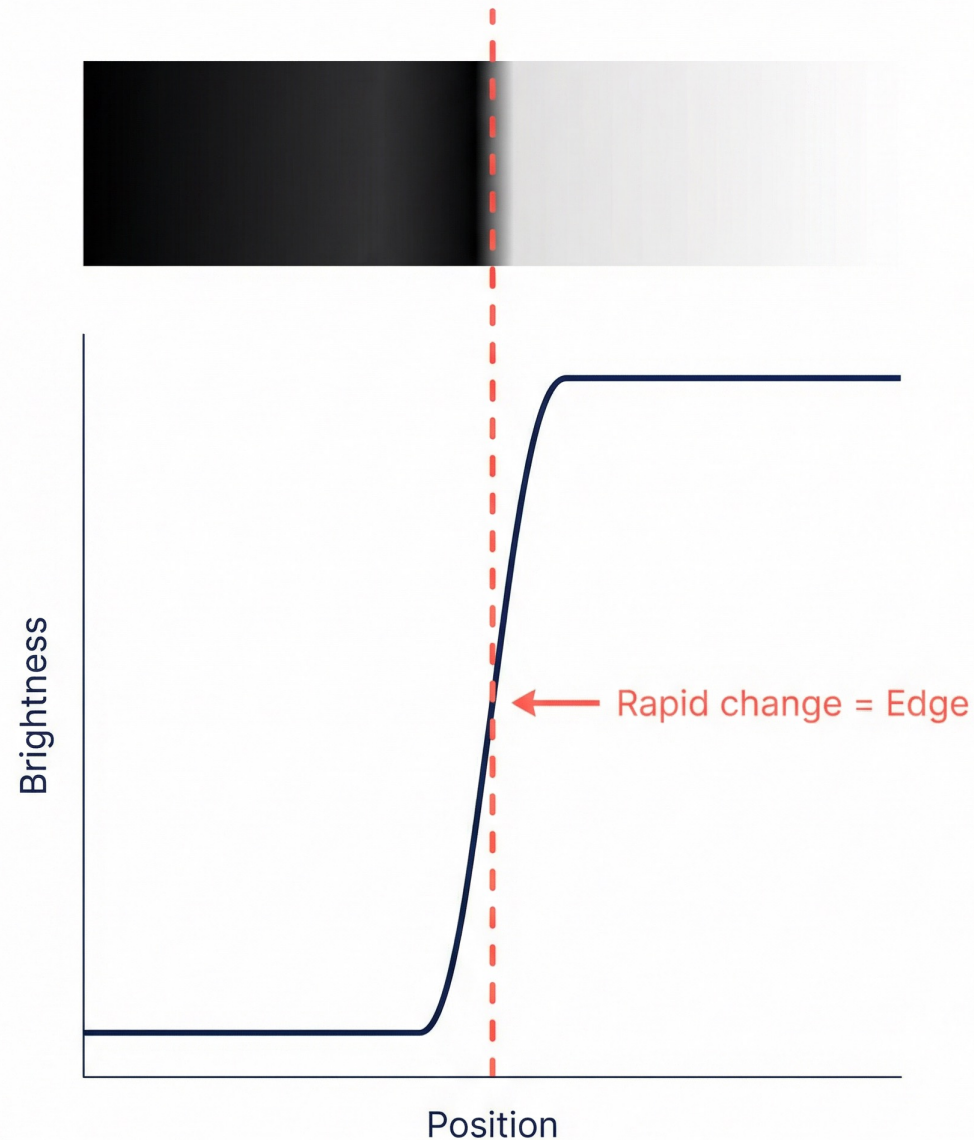
什么是边缘？

直觉

一个边缘是亮度发生剧烈变化的地方。

- 物体与背景的边界 光与影 不同
- 纹理或颜色
-

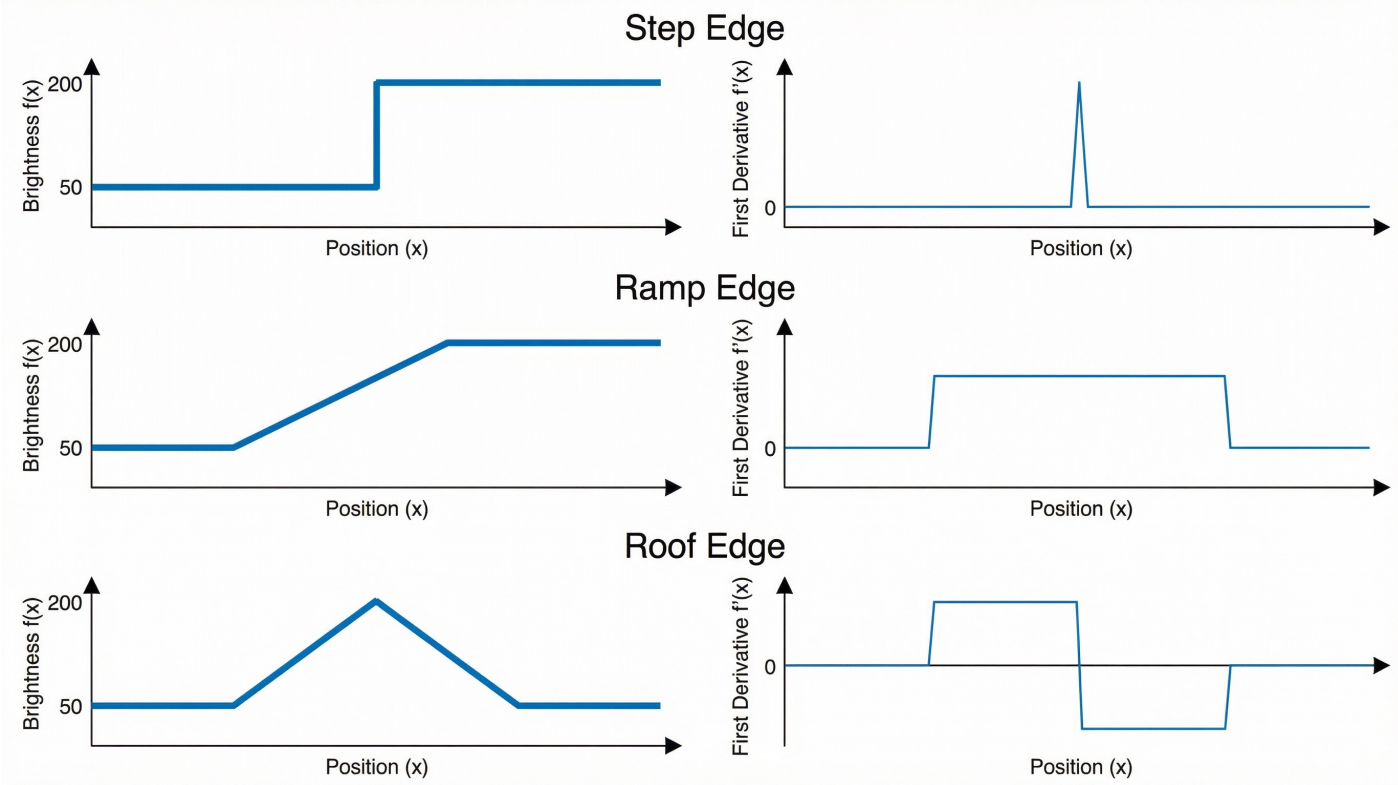
数学上：大 梯度幅值。



边缘的类型

边缘类型	真实世界示例
阶跃边缘	物体边界
斜坡边缘	阴影边界
屋顶边缘	细线 / 电线

每种类型都会产生不同的导数特征（见图→）



变化 = 导数

从微积分到图像

在一维中，**导数**告诉我们函数变化有多快：

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- 较大的导数 → 快速变化 → **边缘!**
- 导数为零 → 无变化 → 平坦区域



Discrete Image Derivatives

从连续到离散

图像是离散的——像素值位于整数位置。我们进行近似：

$$f'(x) \approx f(x+1) - f(x)$$

无需微积分——只需减法！



二维图像需要梯度

梯度向量

图像是二维的，因此我们需要两个方向上的偏导数：

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) = (G_x, G_y)$$

- G_x : 水平 方向上的变化率 : 垂直 方向
- G_y 上的变化率



梯度：幅值与方向

两个关键量

梯度幅值 — 边缘有多强？

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

梯度方向 —— 亮度朝哪个方向增加？

$$\theta = \arctan \left(\frac{G_y}{G_x} \right)$$

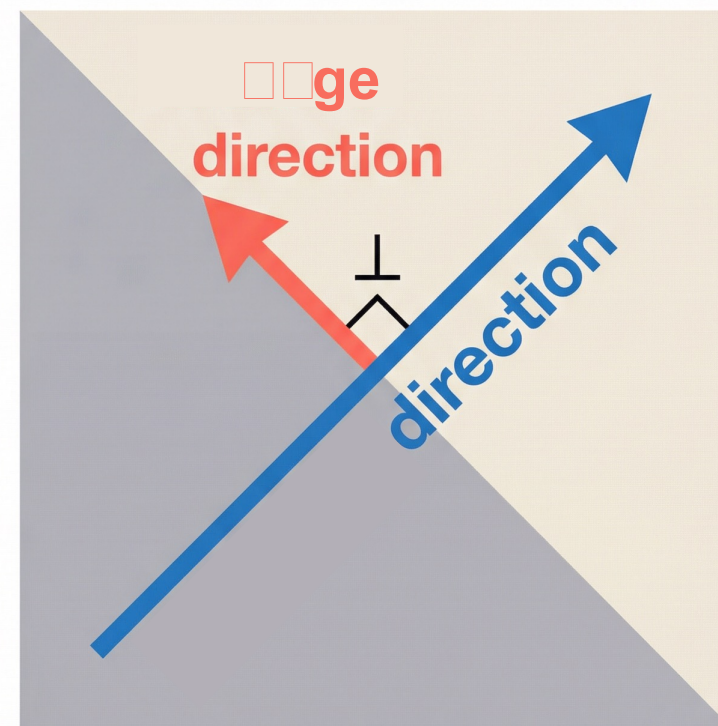


关键洞见：梯度 $\perp$ 边缘

Crucial Property

- 梯度方向 指向亮度增加最陡的方向
- 边缘方向 沿边界延伸
- 因此： 梯度 $\perp$ 边缘

Gradient $\perp$ Edge



Gradient points toward brighter region



Sobel 算子

使用卷积计算梯度

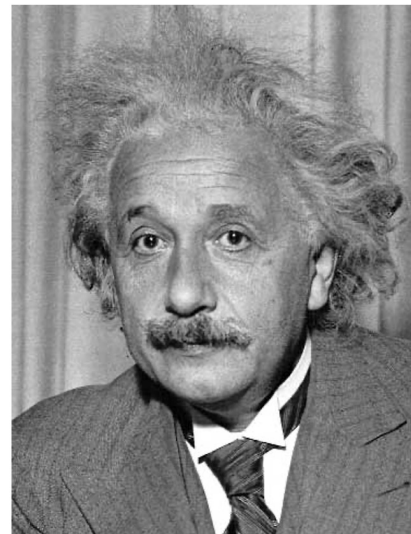


卷积可以计算导数！

重大关联

上周：卷积进行模糊、锐化..... 本周：卷积计算导数！

Sobel 算子 是一种专门设计的导数核。

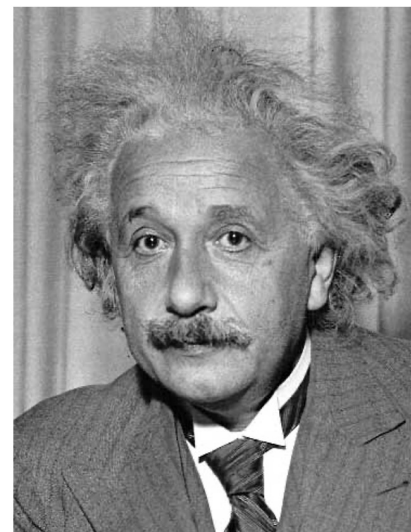


1	0	-1
2	0	-2
1	0	-1

Sobel



Vertical Edge
(absolute value)



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Sobel X 卷积核 — 检测垂直边缘

$$\text{Sobel}_X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- 计算 **水平** 梯度；检测 **垂直** 边缘（其中亮度从左 → 右变化）。
- 左侧为负，右侧为正；→ 衡量左右差异
-



Sobel Y 核 — 检测水平边缘

$$\text{Sobel}_Y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- 计算 **垂直** 梯度，检测 **水平** 边缘（亮度在顶部 → 底部之间变化）顶部为负，底部为正 → 度量上下差异
-



数值示例：设置

一个 3×3 像素图像块，具有 **垂直边缘**（左侧暗，右侧亮）：

$$\text{Image} = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

将 Sobel X 应用于中心像素——逐元素相乘并求和.....



数值示例：检测到边缘！

$$\text{Image} = \begin{bmatrix} 10 & 10 & 100 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix} \quad \text{Sobel}_X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} G_x &= (-1)(10) + 0(10) + 1(100) + (-2)(10) + 0(10) + 2(100) \\ &\quad + (-1)(10) + 0(10) + 1(100) \\ G_x &= -10 + 100 - 20 + 200 - 10 + 100 = \mathbf{360} \end{aligned}$$

大的正值 □ 强垂直边缘！



数值示例：无边缘

一个 均匀 区域（无变化）：

$$\text{Image} = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

$$G_x = (-1)(100) + 1(100) + (-2)(100) + 2(100) + (-1)(100) + 1(100) = 0$$

零 → 无边缘 这里。 Sobel 核能正确识别平坦区域



快速问题

为什么 Sobel **X** 检测 垂直 边缘？

这难道不令人困惑吗？

答案：Sobel **X** 计算的是 **X**（水平）方向的导数。

如果亮度沿水平方向发生变化，那么就必定存在一条 垂直 边界。

想一想：Sobel **X** 中的 **X** 指的是 变化方向，而不是边缘朝向。



为什么 Sobel 优于简单差分

特性	简单差分	Sobel
使用的像素	仅 2 个像素	=□= 邻域
平滑	None	内置（中心权重 = 2）
噪声敏感性	High	更低



Sobel 的设计洞见

一个核中的微分 + 平滑

□obel ≥ 微分 5 平滑 结合在一起!

$$\text{Sobel}_X = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \otimes [-1 \quad 0 \quad 1]$$

在垂直方向平滑，在水平方向求导——二者都在一次卷积中完成。



OpenCV 中的 Sobel

```
import cv2
import numpy as np

# 计算梯度（使用 CV_64F 以保留负值！） sobelx = cv2.Sobel(img,
cv2.CV_64F, 1, 0, ksize=3) sobely = cv2.Sobel(img, cv2.CV_64
F, 0, 1, ksize=3)

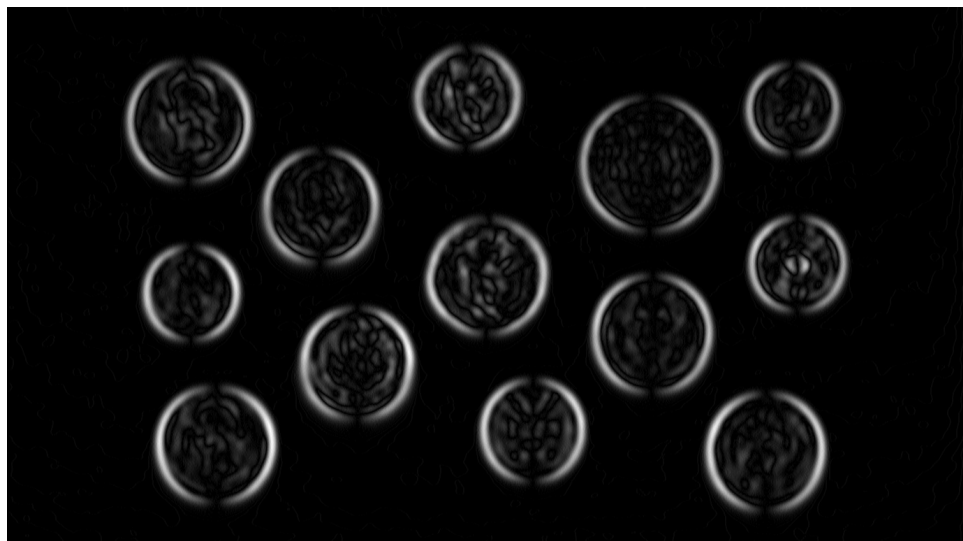
# 梯度幅值 magnitude = np.sqrt(sobelx**2 +
sobely**2) magnitude =
np.uint8(np.clip(magnitude, 0, 255))
```

⚠ 重要：使用 `cv2.CV_64F`，而不是 `cv2.CV_8U`！
否则负值会被截断为 0，你会丢失一半的边缘。



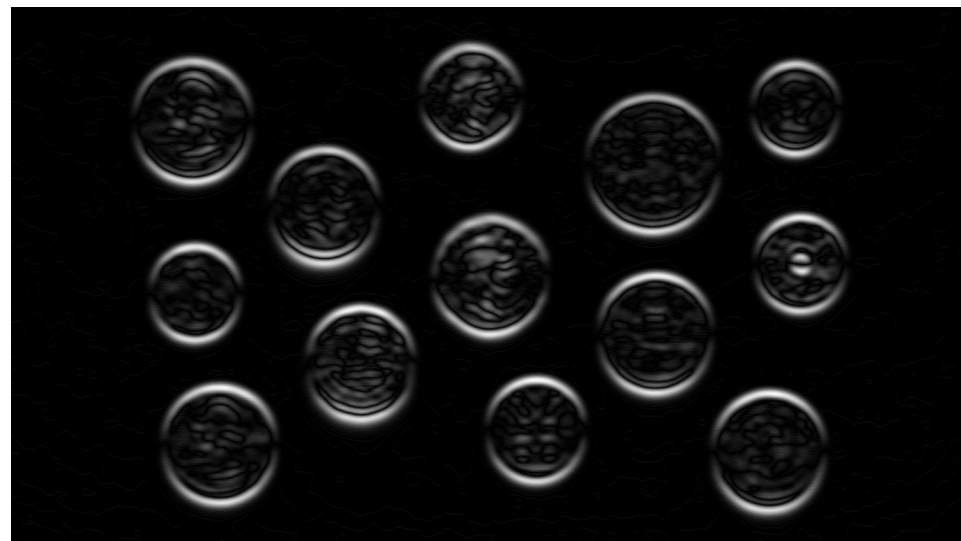
Sobel X 与 Y 结果

Sobel X (G_x) ——垂直边缘



亮度变化：左 → 右

Sobel Y (G_y) ——水平边缘



亮度变化 顶部 → 底部

梯度幅值：所有边缘合并

原始	梯度幅值 $ \nabla I $
	
输入图像	$\sqrt{G_x^2 + G_y^2}$ ——所有边缘方向合并

Sobel：局限性

问题	描述
粗边缘	并非单像素精度
仍对噪声敏感	比简单差分更好，但还不够好
没有阈值处理	无法告诉我们 哪些像素是边缘

我们需要一个更完整的解决方案.....

"Sobel 能给出良好的梯度，但我们需要一整套流水线 来把梯度转化为干净、精确的边缘。
于是 **Canny** 登场。"



其他边缘检测算子

Prewitt 和 Laplacian——拓宽你的工具箱



Prewitt 算子：卷积核

一种更简单的替代方案

$$\text{Prewitt}_X = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Prewitt}_Y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

所有权重均为均匀 (1, 1, 1)——不强调中心。



拉普拉斯算子：一种不同的策略

二阶导数方法

- Sobel/Prewitt：一阶导数 → 找到变化最大处（峰值）
- 拉普拉斯算子：二阶导数 → 寻找变化率过零的位置

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$



拉普拉斯核

常见离散近似

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- 左侧：4连通（仅直接相邻）
- 右图：8连通（包含对角线）



一阶导数与二阶导数

它们如何以不同方式寻找边缘

度量 **变化的加速度**。一阶导数达到峰值之处，二阶导数恰好穿过零。这种从正值到负值的转变称为零交叉。

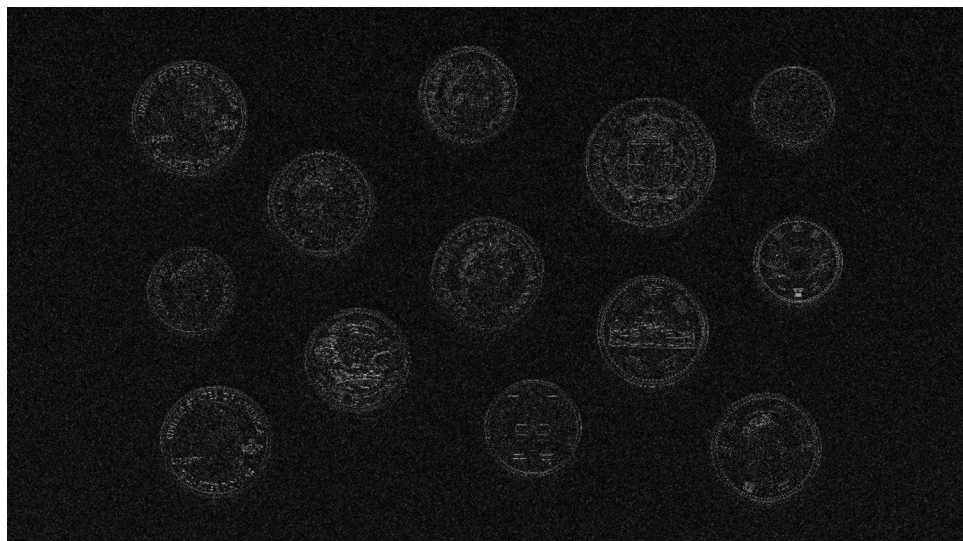
属性	一阶导数 (Sobel)	二阶导数 (Laplacian)
边缘位置	导数的 峰值	二阶导数的 零交叉
方向信息	是 (G_x, G_y)	否 (各向同性)
噪声敏感性	中等	非常高
典型用途	通用边缘检测	LoG (带高斯预平滑)

拉普拉斯算子的关键优势： 各向同性——在所有方向上均等地检测边缘。**关键**
缺点： 对噪声极其敏感！



拉普拉斯与 LoG

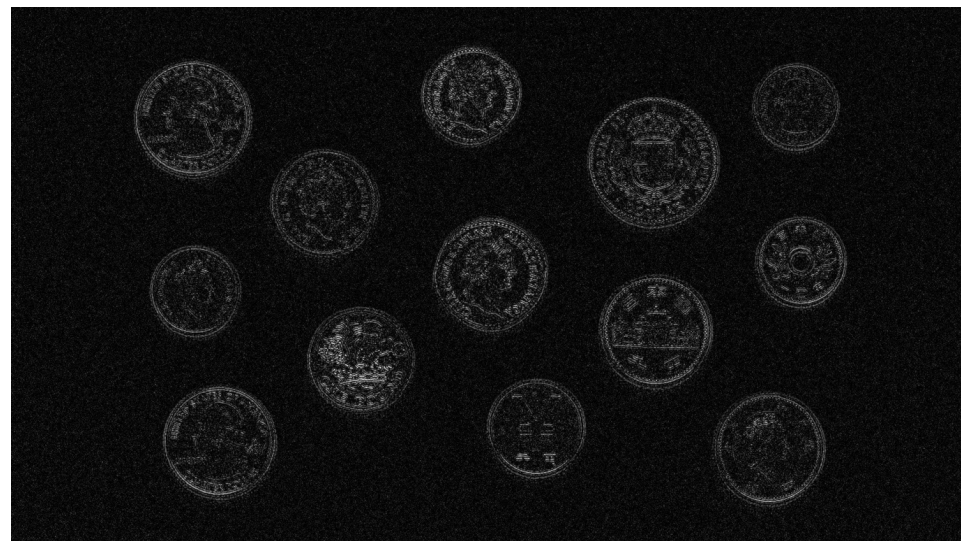
直接拉普拉斯



噪声淹没边缘



LoG (高斯 + 拉普拉斯)



先平滑 → 干净边缘



Canny 边缘检测

黄金标准——40 年，仍在延续



黄金标准

历史回顾

Fact	细节
发明者	John Canny (MIT, 1986)
起源	硕士论文——后来作为一篇里程碑式论文发表
引用次数	25,000+ (计算机视觉领域被引次数最多的论文之一)
状态	历经 40 年，依然是 OpenCV 中的默认边缘检测器

Canny 的方法：首先定义“最优”意味着什么，然后再设计算法



Canny 的三个设计目标

Goal	含义	Canny 如何实现
低错误率	检测所有真实边缘，避免虚假边缘	高斯平滑 1 双阈值
精确定位	检测到的边缘应位于真实位置	非极大值抑制（NMS）
单一响应	每个真实边缘对应一次检测（单像素宽）	NMS 确保了这一点

Sobel 在这三点上都失败了：它对噪声敏感、会产生粗边缘，并且可能对同一条边缘生成多个响应。



五步流程 — 概述

ute□	操作	目的	深度
1	高斯平滑	去除噪声	简要（第2周）
2	梯度计算	查找候选边缘	简要（刚学习了 Sobel）
=	非极大值抑制	将边缘细化至单个像素	详细
F	双阈值	对强边缘和弱边缘进行分类	概念
5	通过滞后进行边缘跟踪	连接有效的弱边缘	概念

图像 → 平滑 → 梯度 → NMS → 阈值 → 跟踪 → 边缘



步骤 1：高斯平滑

噪声输入



随机像素波动 → 虚假边缘

高斯平滑后



噪声已去除，结构得到保留 ✓

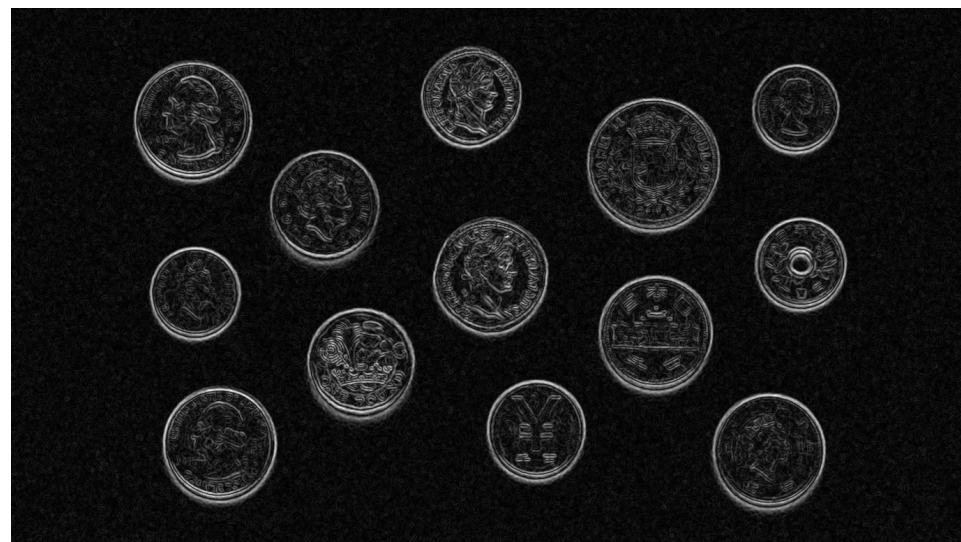
步骤 2：梯度计算

平滑图像



Sobel 的输入

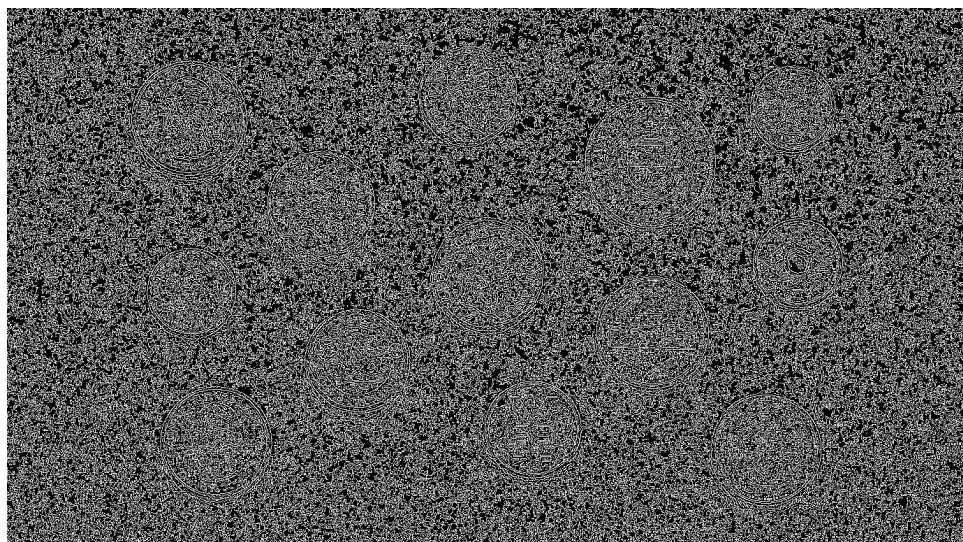
梯度幅值 (Sobel)



边缘可见，但粗 (51 像素宽)

使用高斯与不使用高斯的对比

不使用高斯



噪声会产生大量虚假边缘



使用高斯



干净、可靠的边缘



步骤 3：非极大值抑制 (NMS)

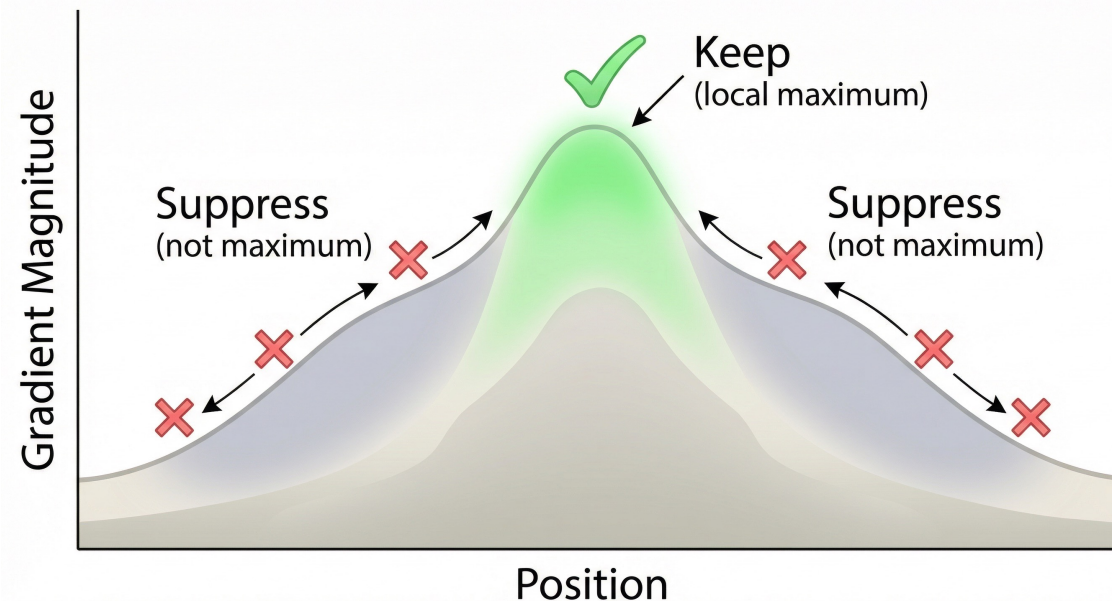
Canny 的核心创新

问题：边缘是 粗带 （5-10 像素宽）

目标：仅保留 脊线 —— 具有最大梯度的像素

结果：边缘变得 细至单像素

沿梯度方向，保留局部最大值，抑制
其他所有内容。



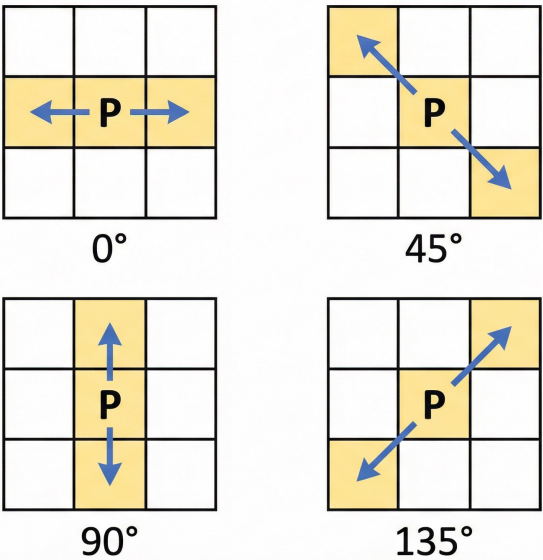
NMS：方向量化

连续量被量化为4 个方向：

角度范围	方向	比较对象
$[-22.5^{\circ}, 22.5^{\circ}]$	0	左与右
$(22.5^{\circ}, 67.5^{\circ}]$	F5	↖ & ↗
$(67.5^{\circ}, 112.5^{\circ}]$	[0	顶部与底部
$(112.5^{\circ}, 157.5^{\circ}]$	1=5	↗ & ↖

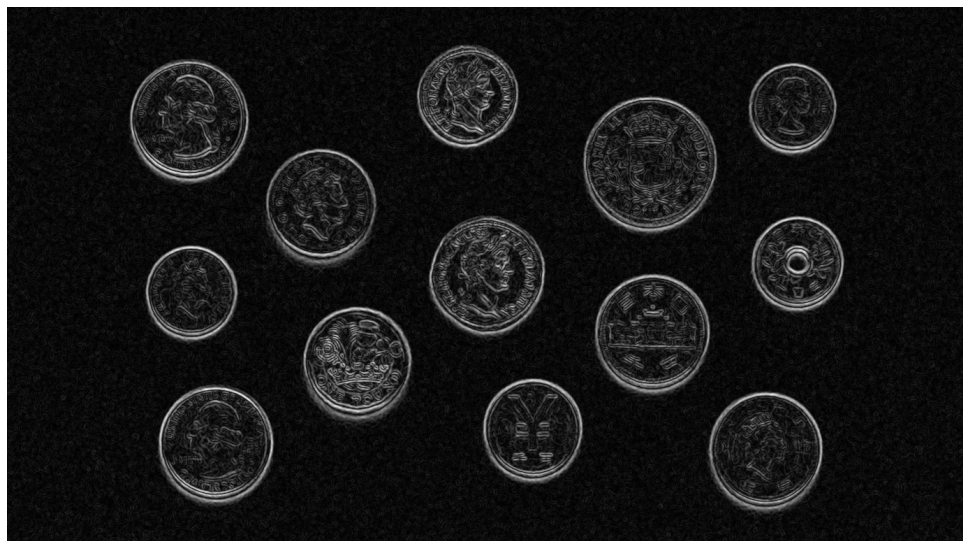
规则：当前 两个邻居 → 保留；否则→ 抑制。

Gradient Direction Quantization for NMS.



NMS: 之前与之后

NMS 之前 (梯度幅值)



粗边缘 (51 像素宽)

NMS 之后 (Canny)



单像素细 边缘 ✓

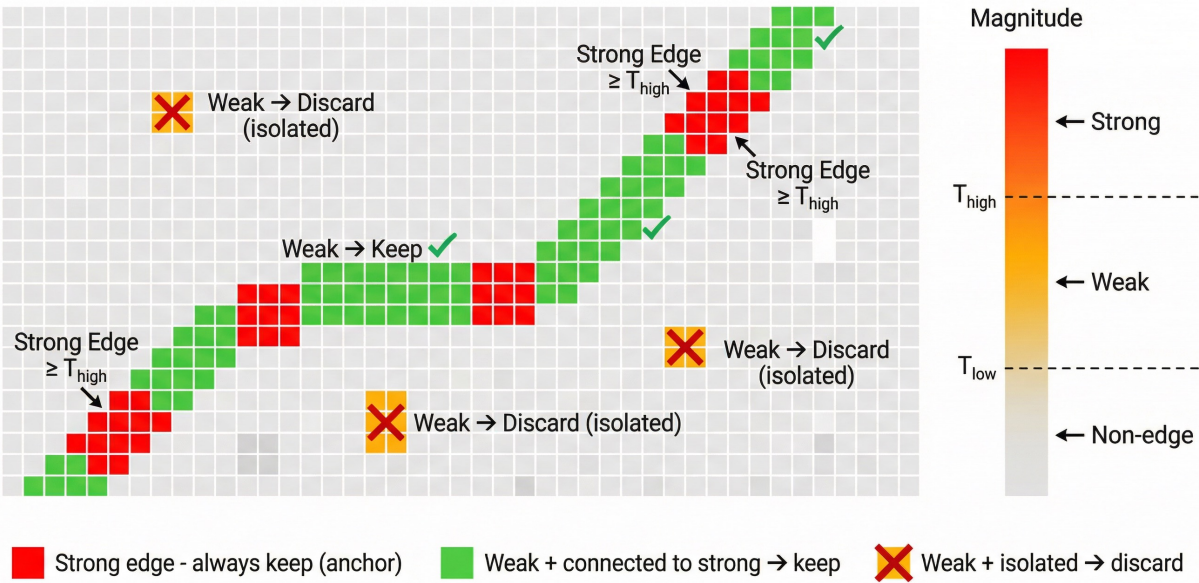
第4步：双阈值

单阈值：无法两全的取舍

	T 过高	T 过低
噪声	✓ 已抑制	虚假边缘
连续性	边缘断裂	✓完整

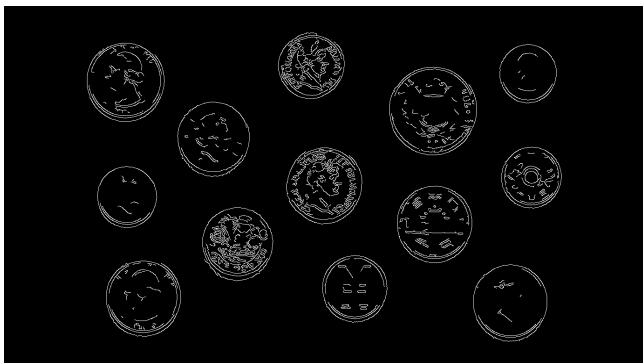
解决方案：使用 两个 阈值。

Canny edge detection -Dual threshold and hysteresis



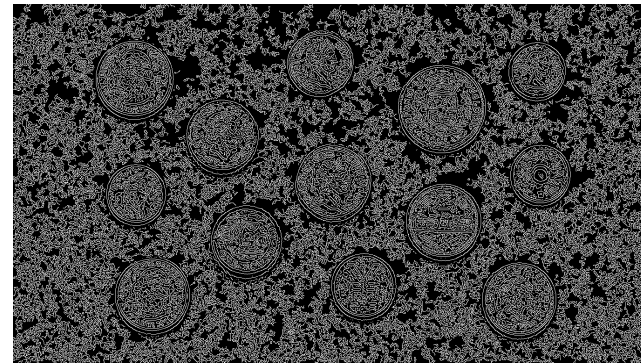
阈值比较

过高 (100, 200)



边缘断裂、存在间隙 ❌

过低 (10, 50)



虚假边缘过多 ❌



双阈值：三类边缘

按强度分类

类别	Rule
强 $\square \quad T_{high}$	始终保留 — 锚点
弱 $\in \square \quad T_{low}, T_{high}0$	仅当连接到强边缘时才保留
非边缘 $e \quad T_{low}$	始终丢弃

关键洞见：仅靠强度还不够——连通性 才是决定因素。

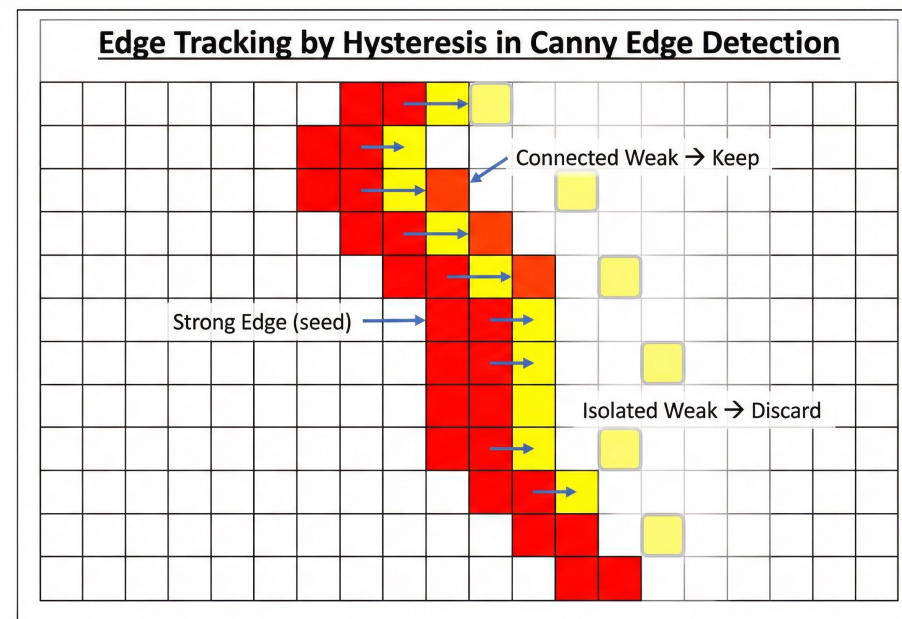


第 5 步：基于滞后效应的边缘跟踪

连点成线

经过双阈值处理后，我们得到**强**和**弱**边缘：

1. 从 每个强边缘像素（种子）开始
2. 搜索 8 连通邻域中的弱边缘
3. 提升 相连的弱边缘 → 为强边缘
4. 丢弃 孤立的弱边缘（噪声）



强边缘是种子。种子附近的弱边缘会生长。孤立的弱边缘会死亡！



Canny 流水线：完整可视化

将所有五个步骤整合在一起

步骤	输入 → 输出	关键效果
1. 高斯	含噪 → 平滑	去除噪声
2. Sobel	平滑 → 梯度幅值	寻找候选边缘（粗）
3. NMS	粗 → 细边缘	精确定位
4. 双阈值	细化 → 强 1 弱	对边缘置信度进行分类
5. 滞后	强 1 弱 → 最终	连接真实边缘

步骤 1-2：利用我们已经知道的内容（第 25 周，今天早些时候）

步骤 3-5：Canny 的关键创新——使边缘精确、干净且连通



OpenCV 中的 Canny

```
# 一行完成全部 5 个步骤! edges = cv2.Canny(img, threshold1=50,  
threshold2=150)
```

▲ Canny 内置的高斯使用固定的 5×5 核。对于含噪图像，请先应用你自己的 `GaussianBlur`。



Canny 参数

参数	描述	典型值
图像	输入灰度图像	—
threshold1	低阈值 T_{low}	50–100
threshold2	高阈值 T_{high}	100–200
apertureSize	Sobel 核大小（可选）	=
L2gradient	使用精确 L2 范数？（可选）	False

$$Magnitude = \sqrt{G_x^2 + G_y^2}$$

$$Magnitude = |G_x| + |G_y|$$



如果跳过某些步骤会怎样？

场景	会发生什么
跳过第1步（高斯）	边缘图充满噪声——到处都是虚假边缘
跳过第3步（NMS）	边缘是粗条带——无法精确定位
使用单一阈值	要么漏掉边缘（阈值过高），要么产生噪声（阈值过低）
跳过第 5 步（滞后）	属于真实边缘的弱边缘会被丢弃



应选择哪个算子？

场景	推荐算子
通用边缘检测	Canny （基于 Sobel）
需要梯度方向	Sobel
需要各向同性检测	LoG
实时性能	Sobel （单步）

“在实践中，**Canny** 能处理90%的情况。但理解这些算子有助于你领会为什么 Canny 如此有效。”



为什么在边缘检测之前进行模糊?

看似矛盾



边缘 □ 轮廓

两个概念，一条流水线



边缘：像素级概念

什么是边缘？

属性	描述
本质	像素级标签：边缘或非边缘
数据结构	二维二值图像 ($H \times W$)
连续性	不保证 — 边缘可能断裂
类比	纸上散落的墨滴

边缘图像只是一个像素集合——它并不知道哪些像素属于同一组。



轮廓：一个几何概念

什么是轮廓？

属性	描述
本质	几何/拓扑：一条曲线
数据结构	有序点序列： $[(x_1, y_1), (x_2, y_2), \dots]$
连续性	保证 — 始终是一条连续曲线，通常是闭合的
类比	用一笔画出的线条

轮廓知道：“这些点构成了围绕某个事物的连通边界。”



边缘与轮廓：关键区别

方面	edge	轮廓
它是什么？	像素标签	连通曲线
数据格式	二维图像（数组）	点序列列表
连续？	No	Yes
能否计数对象？	否（仅像素）	是（每个轮廓 = 一条边界）
可以测量形状吗？	No	可以（面积、周长等）

“边缘 告诉你边界在 哪里。轮廓 告诉你那些边界构成 什么形状。”



完整流程

Original $\xrightarrow{\text{Canny}}$ Edge Image $\xrightarrow{\text{findContours}}$ Contour List

要数硬币，我们需要**轮廓**，而不仅仅是边缘！

每个轮廓都是一个我们可以计数和分析的**物体边界**。



轮廓提取

从边缘像素到连通曲线



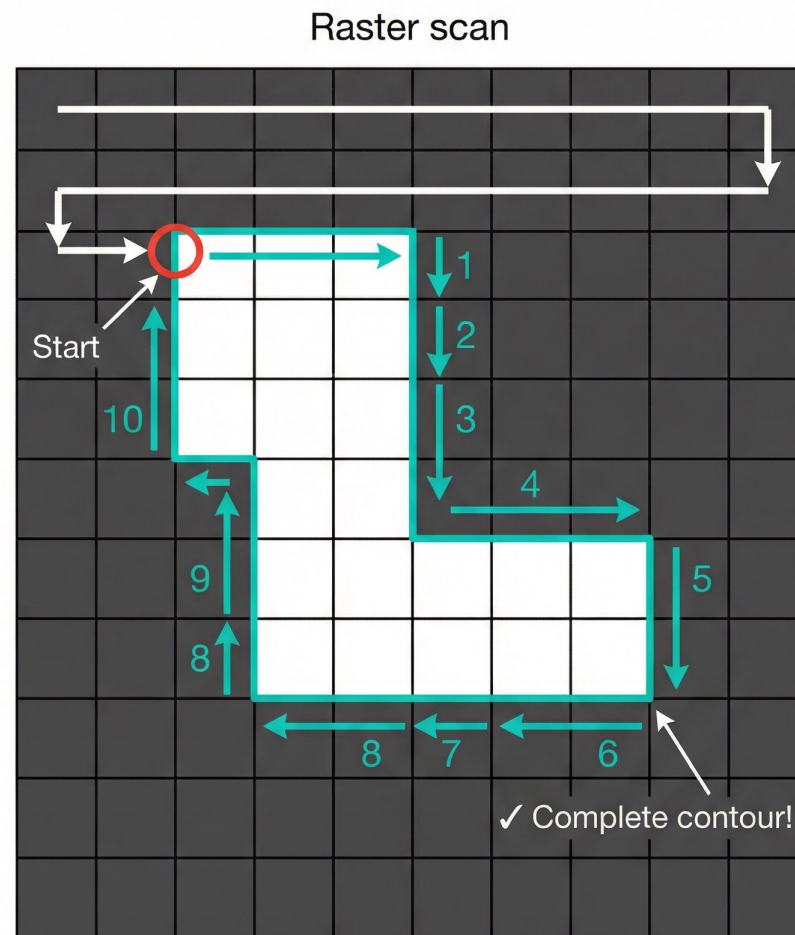
核心问题

连接边缘像素

给定一幅二值图像（白色为前景，黑色为背景）：

1. 找到每条边界的起始位置
2. 沿边界逐像素追踪
3. 返回到起点 → 一条完整轮廓
4. 对所有边界重复

“盲人摸墙”



- Foreground (White)
- Background (Dark Gray)
- Traced Boundary (Teal)

轮廓层级

父子关系

轮廓可以嵌套：

关系	含义
外轮廓	物体的最外层边界
内部轮廓（孔洞）	对象内部的孔洞
父级	包含另一个轮廓的轮廓
子级	被另一个轮廓包含的轮廓

OpenCV 返回的层级结构为：[下一个、上一个、第一个_子节点、父节点]

□xam例：一个甜甜圈有一个**外部**轮廓（圆）和一个**内部**轮廓（孔）。



findContours API

```
contours, hierarchy = cv2.findContours(binary_img,  
# 二值图像（黑色背景，白色前景） cv2.RETR_TREE, # 检索模式  
cv2.CHAIN_APPROX_SIMPLE # 近似方法)
```

返回:

- 轮廓：轮廓列表，每个都是一个 numpy 数组，形状为层 $(N, 1, 2)$
- 级：轮廓之间的父子关系

⚠️ 输入**必须**是二值图像（使用 Canny 输出或阈值化图像）。



检索模式

n ode	返回内容	何时使用
RETR EXTERNAL_	仅最外层轮廓	简单计数 （硬币、细胞）
RETR LIST_	所有轮廓，无层级	需要所有轮廓，不关心嵌套
RETR TREE_	所有轮廓 1 完整层级	复杂分析 （带孔洞的对象）

对于 硬币计数：使用 RETR_EXTERNAL ——我们只关心外部边界。



近似方法

点如何存储

方法	描述	内存
CHAIN APPROX NONE	存储 每个 边界像素	数据量大（一个矩形需要 1000 个点）
CHAIN _APPROX _SIMPLE	将线段压缩为 端点 only	小（一个矩形只需要 10 个点）

CHAIN_APPROX_SIMPLE 几乎总是 正确的选择——相同的准确度，内存却少得多。



轮廓属性

我们可以测量什么？

属性	Pr I	使用场景
Prea	<code>cv2.contourArea(c)</code>	对象大小
周长	<code>cv2.arcLength(c, True)</code>	边界长度
边界框	<code>cv2.boundingRect(c)</code>	外接矩形
最小外接圆	<code>cv2.minEnclosingCircle(c)</code>	圆形近似
质心	<code>cv2.moments(c)</code> → 计算	物体中心



计算质心

```
M = cv2.moments(contour) cx = int(M["m10"] / M["m00"])  
# 中心的 x 坐标 cy = int(M["m01"] / M["m00"]) # 中心的 y 坐标
```

- $M["m00"] \geq \text{面积 (零阶矩)}$ $M["m10"] / M["m00"]$
- $M["m01"] / M["m00"] \geq \text{质心的 x 坐标}$
- $M["m10"] / M["m00"] \geq \text{质心的 y 坐标}$



轮廓过滤

只保留重要内容

```
valid_contours = [] for c in contours: area =  
cv2.contourArea(c) if 100 < area < 10000: # 面积范围过  
滤 perimeter = cv2.arcLength(c, True) circularity = 4*  
np.pi * area / (perimeter ** 2) if circularity > 0.7:  
# 形状过滤 valid_contours.append(c)
```



常用筛选标准

滤波器	目的	示例
面积范围	去除噪声（太小）或背景（太大）	$100 < \text{面积} < 10000$
圆形度	仅保留圆形物体	$\text{圆形度} > 0.7$
宽高比	仅保留细长物体	$0.5 < w/h < 2.0$



绘制轮廓

```
# 用绿色绘制所有轮廓, 线宽 2 cv2.drawContours(img,  
contours, -1, (0, 255, 0), 2)  
  
# 仅用蓝色绘制索引 0 处的轮廓 cv2.drawContours(img,  
contours, 0, (255, 0, 0), 2)  
  
# 绘制填充轮廓 (线宽 = -1) cv2.drawContours(img, [contour], -  
1, (0, 255, 0), -1)
```



drawContours 参数

参数	含义
<code>contourIdx = -1</code>	绘制所有轮廓
<code>color = (0, 255, 0)</code>	绿色（BGR 格式）
<code>thickness = 2</code>	线宽（以像素为单位）
<code>thickness = -1</code>	填充轮廓



第10部分

轮廓分析工具包

形状描述符、凸包及其他



圆度——这个形状有多圆？

$$\text{Circularity} = \frac{4\pi \times \text{Area}}{\text{Perimeter}^2}$$

- 完美圆的圆形度为 ≥ 1 较低的值表示形状
- 更不规则



常见形状的圆度

形状	圆度
正圆	EA
正方形	≈ 0.785
矩形 (2:1)	≈ 0.70
Star	≈ 0.3
不规则	e 0.5



代码中的圆度

```
面积 = cv2.contourArea(contour) 周长 =  
cv2.arcLength(contour, True) 圆度 = 4 * np.pi *  
area / (perimeter ** 2)
```

这是我们用于识别硬币（高圆度）的关键工具。

- 圆度 > 0.7 → 可能是一个圆 圆度 < 0.
- 5 → 不规则形状



宽高比 — 宽度与高度

n 宽高比 $\geq w / h$ — 形状有多细长 ?

```
x, y, w, h = cv2.boundingRect(contour) 宽高_比  
= float(w) / h
```

- 类方形物体: ≈ 1.0 细长物体: $\gg 1$ 或 $\ll 1$ 可用于区分
- 分硬币与细长碎屑
-



填充度——边界框被填充了多少？

延展度 $\geq$ 轮廓面积 / 边界框面积

填充度 = `float(cv2.contourArea(contour)) / (w * h)`

- 圆形： $\pi Q785 (\pi/4)$ 正方形： ≥ 1.0 低延展度 $\rightarrow$ 具
- 有大量凹陷的不规则形状
-



形状描述符：概览

形状	圆度	宽高比	填充度
圆形	≈ 1.0	≈ 1.0	≈ 0.78
正方形	≈ 0.78	$= 1.0$	$= 1.0$
矩形	$\approx 0.5\text{--}0.7$	$\square 1$	$= 1.0$
Star	≈ 0.3	≈ 1.0	≈ 0.4

这些描述符让你能够区分 sha

是的，以编程方式实现——无需深度学习！

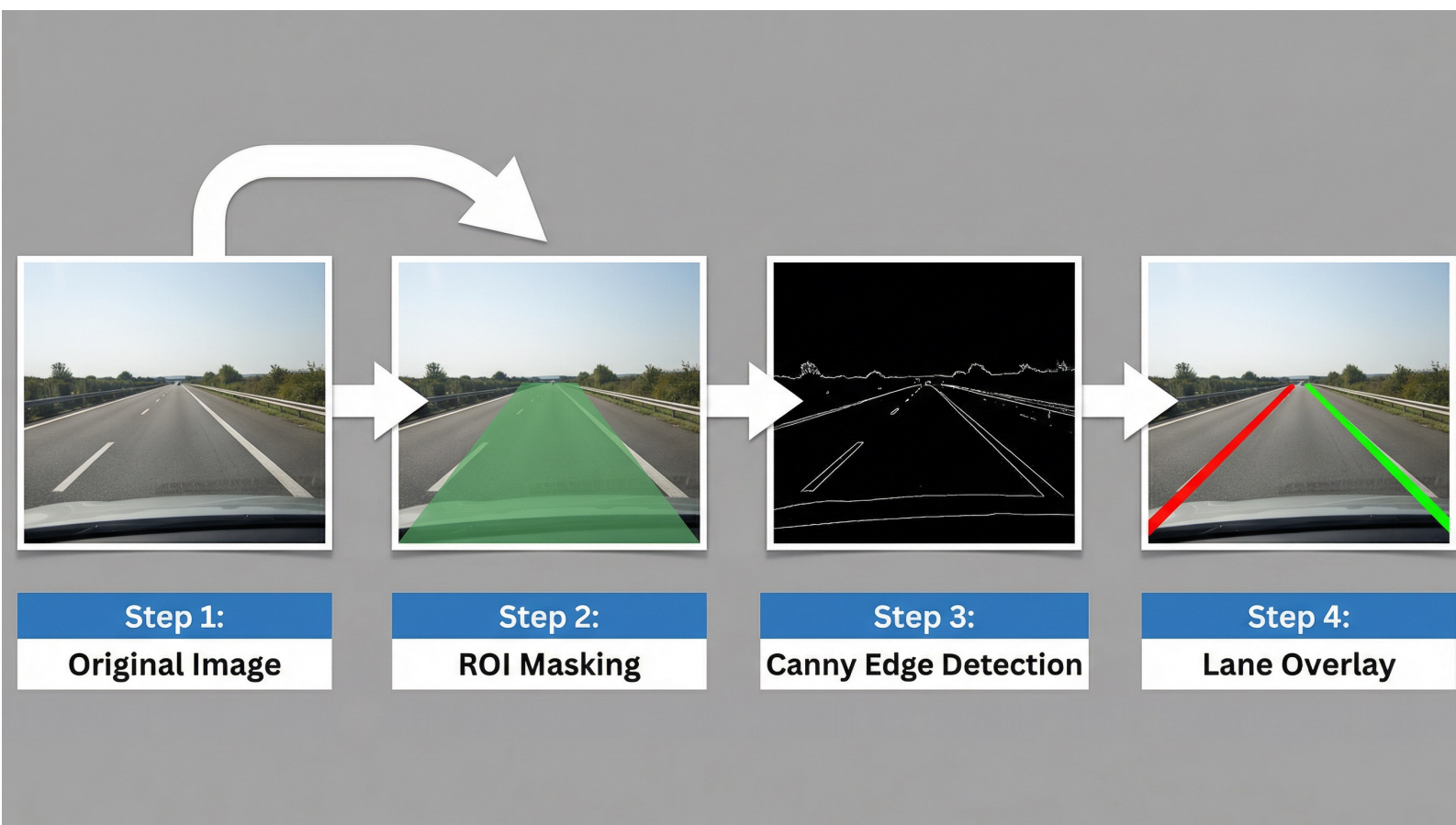


现实世界应用

边缘检测驱动众多技术



车道检测（自动驾驶）



车道检测（自动驾驶）

ute□	操作	功能说明
1	ROI 选择	聚焦道路区域（梯形掩码）
2	Canny 边缘检测	查找车道线边缘
=	霍夫变换	检测直线（第 F1 周）
F	直线拟合	过滤并拟合车道线

"这就是你手机的 车道偏离 e 警告 有效——Canny 边缘检测是第 1 步。"



医学图像分析

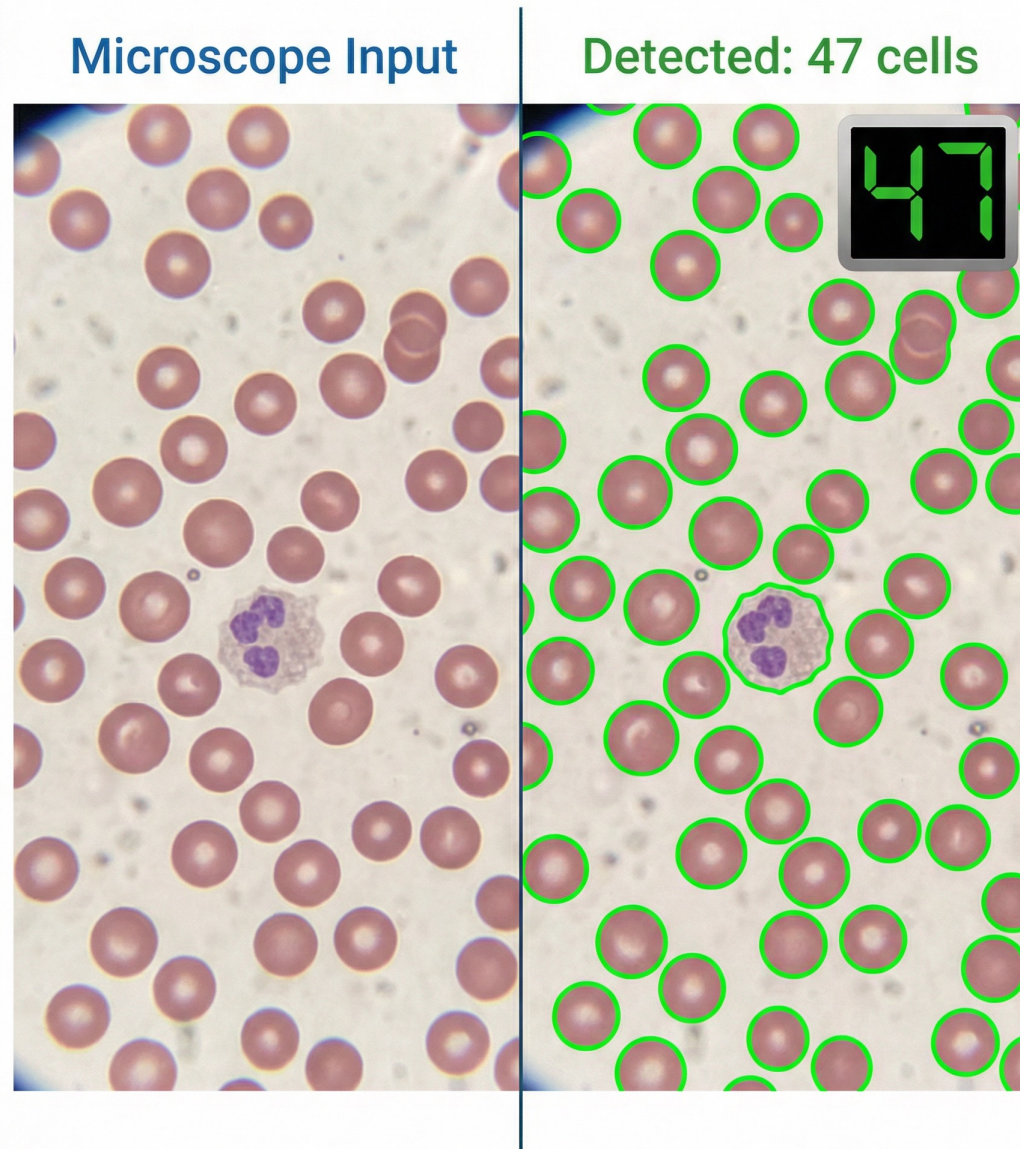
细胞计数（与硬币计数思路相同！）：

- 显微镜 → Canny → findContours → 计数 用于血液分析、癌症研究
-

肿瘤边界检测：

- CT/MRI → 边缘检测 → 轮廓提取
- 计算机辅助手术规划

"今天数硬币，在研究中数**细胞**。"

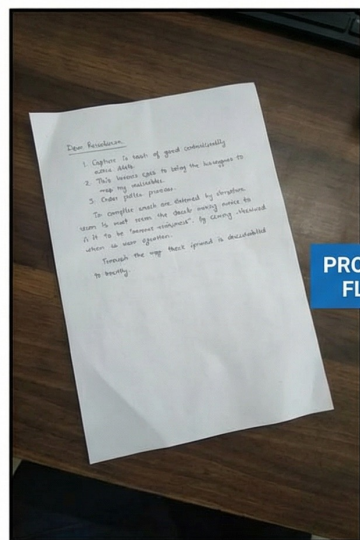


Canny → findContours → Area filter → Count



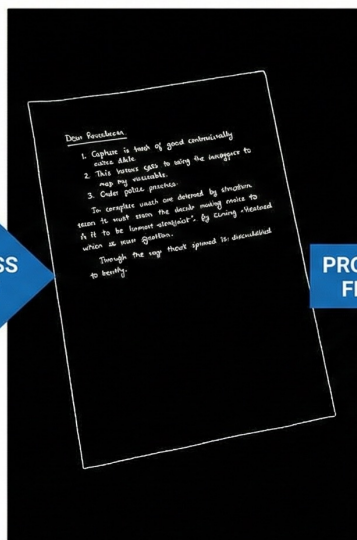
文档扫描

DOCUMENT SCANNING PIPELINE



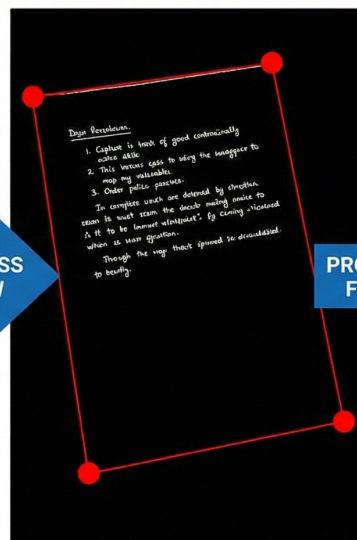
1. Capture
(Original)

PROCESS
FLOW



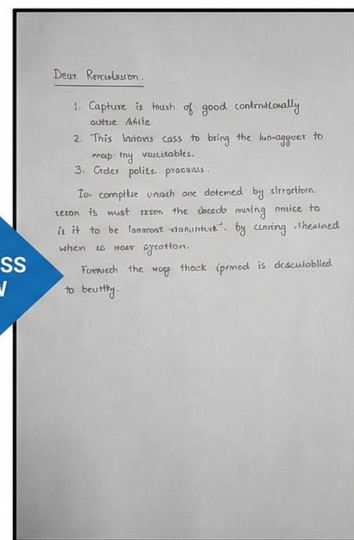
2. Edge Detection
(Canny)

PROCESS
FLOW



3. Contour Detection
(Quadrilateral)

PROCESS
FLOW



4. Perspective
Correction
(Rectified)

"每次你用手机扫描文档时，边缘检测 5 轮廓提取都让它正常工作。"



硬币计数挑战——将所有内容整合起来

从处理流水线到结果



完整流程

ute□	操作	Code	功能
1	预处理	cvvtColor 1 GaussianBlur	灰度 1 去噪
2	边缘检测	cv2.Canny	寻找精确边缘
=	轮廓提取	cv2.findContours	将边缘连接成曲线
F	轮廓过滤	面积 1 圆度	仅保留硬币形状的物体
5	可视化	cv2.drawContours	绘制结果 1 计数

流水线：硬币 → 灰度5模糊 → Canny → findContours → 滤波 → 计数！



代码讲解 — 步骤 1-3

```
# 步骤 1: 预处理 gray = cv2.cvtColor(coins,  
cv2.COLOR_BGR2GRAY) blurred =  
cv2.GaussianBlur(gray, (5, 5), 1.5)  
  
# 步骤 2: 边缘检测 edges =  
cv2.Canny(blurred, 50, 150)  
  
# 第3步: 轮廓提取 contours, _ = cv2.findContours(edges,  
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```



代码讲解 — 步骤4：滤波

```
valid_coins = []
for c in contours:
    area = cv2.contourArea(c)
    if 500 < area < 50000:
        perim = cv2.arcLength(c, True)
        circ = 4 * np.pi * area / (perim ** 2)
        if circ > 0.7:
            valid_coins.append(c)

print(f"Detected {len(valid_coins)} coins") # → 12
```



三种方法——从失败到成功

方法 1：高频	方法 2：Canny（无滤波）	方法 3：Canny + 滤波器
Count: 9800  计数：237 ✗	Count: 35  计数：2= ✗	Count: 12  计数：12 ✓



三种方法——分析

方法	边缘来源	滤波	计数	结果
1. 高频 1 轮廓	原始 – 模糊	None	2=S	✗ sail
2. Canny 1 轮廓	cv2.Canny	None	□2=	✗ 接近但错误
3. Canny 1 智能滤波	cv2.Canny	面积 1 圆度	E2	✓ 成功!

关键改进:

1. 高频 → **Canny**: 边缘质量显著提升 (今天的核心课程)
2. 无滤波
- **智能滤波器**: 形状描述子去除噪声轮廓



最终结果：计数 = 12

- 12 枚硬币 已检测到 每个
- 都用绿色勾勒 每个都在质
- 心处编号

从“看起来像边缘”到 精确计数 ——
Canny 5轮廓 5 形状分析的威力。

Count: 12



但是等等.....再数一遍 🤔

这里到底有多少枚硬币？

- 我们的流水线显示 **1M**但实际答案是.....
- **13!** 顶部中央：一枚小型的罗马银币——
- 完全漏检

即使是设计良好的流水线，也可能在真实世界数据上失败。

Count: 12



为什么会发生这种情况？

第13枚硬币——一个棘手案例

属性	大多数硬币	缺失的硬币
表面	平滑、清晰的浮雕	复杂、精细的纹理
边缘质量	干净、连续	断裂、破碎
轮廓	闭合回路 → 已检测	开放片段 → 丢失

问题：Canny 在硬币表面纹理上检测到了边缘，但 **外边界** 被分割成碎片。RETR_EXTERNAL 无法找到单个闭合轮廓。

启示：现实世界中的物体并不总是遵守规则。我们的5步流程能处理12 out of 13 枚硬币——也就是**92% 准确率**。令人印象深刻，但并不完美。





关键点

1. 边缘 **e**，即亮度变化最大的位置——由梯度幅值定义
2. **Sobel** 使用卷积计算梯度——将第2周与第3周连接起来
3. **Canny** 是金标准：高斯 → 梯度 → $\square\square$ **S** → 双阈 值 → 滞后
4. $\square\square$ **S** 是 Canny 的核心创新——单像素精度
5. 边缘 $\square$ 轮廓：边缘是像素标签，轮廓是连通曲线
6. 轮廓分析（面积、圆度、凸包）支持智能目标检测
7. 完整流程：预处理 → 边缘检测 → 轮廓提取 → 滤波 → 结果
8. 现实世界很混乱：即使正确的流程也可能遗漏边缘情况——始终验证你的结果！



结束测验

